

PREDICTION OF CYBER ATTACKS USING DATA SCIENCE TECHNIQUE

**Thesis Submitted to RAJENDRA UNIVERSITY for the award of the degree of Bachelor
of Computer Applications**

By

Ayush Das

(Roll no. R05423BCA009)

Department of BCA, YCAT, Sakma, Balangir

Under the Supervision of

Dr. Jagamohan Padhi

(Guide)

Associate Professor,

Principal, YCAT (Professional), Sakma, Balangir



RAJENDRA UNIVERSITY

PRAJNA VIHAR, BALANGIR, ODISHA-767002

Year 2026

CERTIFICATE

This is to certify that the thesis titled “ **PREDICTION OF CYBER ATTACKS USING DATA SCIENCE TECHNIQUE** “, submitted to Yuvodaya College of Advanced Technology, Sakma, Balangir affiliated to Rajendra University, Odisha , India by **Ayush Das**, Roll No. R05423BCA009 for the award of the degree of Bachelor of Computer Applications is a Bonafide record of research work carried out by her under my supervision and guidance.

The candidate has fulfilled all the requirements.

The thesis, which is based on the Candidate’s own work, has not been submitted elsewhere for a degree/diploma

In my opinion, the thesis is a standard requirement for a bachelors degree.

External

Dr. Jagmohan Padhi
Associate Professor,
Principal,
YCAT, Sakma, Balangir
Odisha 767020 (INDIA)

DECLARATION

I certify that

- a) The work contained in this thesis is original and has been done by myself under the supervision of my supervisor.
- b) This work has not been submitted to any other institute for any degree or diploma.
- c) I have confirmed the norms and guidelines provided in the Ethical code of Conduct of the Rajendra University
- d) Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references.
- e) Whenever I quote written materials from other sources, credit is given to the sources by citing them.
- f) From the plagiarism test , it was found that the similarity index of the whole thesis is less than 15% as per the University Grant Commission guidelines, which is within 25 % and less than 10% from a single paper as per the university guidelines.

Date:

Place:

Mr. Ayush Das
Roll No.: R05423BCA009

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, **Dr. Jagamohan Padhi**, for their invaluable guidance, constant encouragement, and unwavering support throughout the course of this research. Their insightful feedback and expertise have been instrumental in shaping this thesis, and I am deeply grateful for their patience and dedication.

I would also like to extend my heartfelt thanks to **Dr. Jagamohan Padhi**, the Principal of **Yuvodaya College of Advanced Technology, Sakma , Balangir**, for his continuous support and encouragement. His leadership and commitment to academic excellence have greatly contributed to the successful completion of this work.

A special note of appreciation goes to my friends and peers, who have always been there to offer support, encouragement, and constructive criticism. Their companionship and insights have made this journey more enriching and memorable.

I would also like to acknowledge the **Yuvodaya College of Advanced Technology, Sakma , Balangir** for providing the necessary resources, facilities, and academic environment that facilitated the completion of this research. The access to library resources, research materials, and technical support was critical to the success of this study.

Finally, I wish to thank my family for their unconditional love, support, and understanding throughout this entire process. Without their belief in me, this work would not have been possible.

ABSTRACT

In the context of a digitally transforming world where interconnectivity of networks is rapidly expanding, the threat of cyber-attacks, including ransomware, state-sponsored cyber-attacks, etc., is increasing exponentially in terms of frequency, complexity, and variety. Cybersecurity has emerged as one of the pillars of digital resilience. This thesis explores the synergistic potential of integrating data science methodologies as well as supervised machine learning approaches for the development of proactive predictive cyber threat detection and classification systems, beyond the limitations of reactive approaches, which are proven inadequate in coping with the dynamic, non-linear, and high-dimensional nature of network traffic patterns.

In light of the limitations of existing approaches, including feature complexity, generality, and high false positives, the thesis treats intrusion detection as a multi-class classification problem. The approach involves a rigorous data analysis pipeline including data preprocessing, feature analysis, anomaly detection, as well as hyperparameter tuning. Four machine learning approaches, namely Logistic Regression, Decision Tree, Random Forest, and Support Vector Classifier, are systematically compared using the NSL KDD dataset. Performance is evaluated using rigorous metrics including accuracy, precision, recall, as well as F1-score, while placing greater emphasis on real-time adaptability as well as low false alarms.

In the context of the proposed approach, the intrusions are classified into four main categories: Denial of Service (DoS), Remote-to-Local (R2L), User-to-Root (U2R), as well as Probe attacks. The experimental findings substantiate the efficacy of the suggested methodology; specifically, the refined machine learning models successfully discern patterns within the training dataset, thus yielding enhanced predictive accuracy relative to conventional methods. This approach incorporates data science techniques alongside evolutionary optimization principles, consequently facilitating the creation of intelligent, self-evolving intrusion detection systems, which possess the capacity to proactively forecast cyber threats.

In conclusion, the proposed approach contributes to the development of data-driven cybersecurity architectures, empowering organizations to proactively defend themselves against cyber-attacks, thereby promoting a more secure as well as trustworthy digital environment in the context of an increasingly interconnected world.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	
	LIST OF FIGURES	
	LIST OF TABLES	
1	INTRODUCTION	1
	1.1 Data Science	1
	1.1.1 Data Scientist	1
	1.2 Artificial Intelligence	2
	1.3 Natural Language Processing	2
	1.4 Machine Learning	3
	1.5 Objectives	4
	1.6 Project Goals	4
2	LITERATURE SURVEY	
	2.1 Literature Survey	5
3	AIM AND SCOPE OF THE INVESTIGATION	
	3.1 Project Proposal	10
	3.2 Scope of the Project	10
	3.3 Existing System	10
	3.3.1 Disadvantages	11
	3.4 Preparing Dataset	11
	3.5 System Study	11
	3.5.1 Classification of Attacks	11
	3.5.2 Attacks Summary	12
	3.6 Feasibility Study	16
	3.6.1 Data Wrangling	16
	3.6.2 Data Collection	16
	3.6.3 Preprocessing	16

3.6.4 Building the Classification model	16
3.6.5 Construction of a Predictive model	16
4 EXPERIMENTAL OR MATERIALS AND METHODS, ALGORITHMS USED	
4.1 Project Requirements	17
4.1.1 Functional Requirements	17
4.1.2 Non-Functional Requirements	17
4.1.3 Environment Requirements	17
4.2 Software Description	18
4.2.1 Anaconda Navigator	18
4.2.2 Jupyter Notebook	19
4.2.3 Python	20
4.3 Python Libraries Needed	22
4.3.1 NUMPY Library	23
4.3.2 PANDAS Library	23
4.3.3 MATPLOTLIB Library	23
4.3.4 SCIKIT-LEARN Library	23
4.3.5 TKINTER	24
4.4 System Architecture	24
4.5 UML Diagrams	24
4.5.1 Class Diagram	24
4.5.2 Use Case Diagram	25
4.5.3 Workflow Diagram	26
4.5.4 Activity Diagram	26
4.5.5 Sequence Diagram	27
4.5.6 Entity Relationship Diagram	27
4.6 Algorithms Used	28
4.7 Comparing Algorithm with prediction in the form of best Accuracy result	32
4.8 Module Description	36
4.8.1 Data Validation process and Visualization	36
4.8.2 Prediction of DOS Attacks	40
4.8.3 Prediction of R2L Attacks	41
4.8.4 Prediction of U2R Attacks	42

	4.8.5 Prediction of Probe Attacks	42
	4.8.6 Prediction of Overall Network Attacks	43
	4.8.7 GUI based prediction results of Network Attacks	44
5	RESULTS AND DISCUSSION, PERFORMANCE ANALYSIS	
	5.1 Performance Analysis	45
	5.2 Discussion	47
6	SUMMARY AND CONCLUSION	
	6.1 Summary	48
	6.2 Conclusion	48
	6.3 Future Work	48
	REFERENCES	49

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
1.1	Process of Machine Learning	3
3.1	Process of data flow Diagram	16
4.1	Anaconda Navigator Interface	19
4.2	System Architecture	24
4.3	Class Diagram	25
4.4	Use Case Diagram	25
4.5	Workflow Diagram	26
4.6	Activity Diagram	26
4.7	Sequence Diagram	27

4.8	Entity Relationship Diagram	27
4.9	Logistic Regression	29
4.10	Decision Tree Classifier	30
4.11	Random Forest Classifier	31
4.12	Support Vector Classifier	31
4.13	Data Frame	36
4.14	Percentage level of protocol type	37
4.15	Comparison of service type and protocol type	38
4.16	Data Validation	40
4.17	Prediction of DOS Attacks	41
4.18	Prediction of R2L Attacks	41
4.19	Prediction of U2R Attacks	42
4.20	Prediction of Probe Attacks	43
4.21	GUI based prediction	45
5.1	Accuracy comparison of DOS Attack	45
5.2	Accuracy comparison of R2L Attack	46
5.3	Accuracy comparison of U2R Attack	46
5.4	Accuracy comparison of Probe Attack	46
5.5	Accuracy comparison of Overall Attack	47

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
3.1	Attack Types Grouped to respective class	13
3.2	Description of Attacks	13

CHAPTER-1

INTRODUCTION

DOMAIN OVERVIEW

1.1 DATA SCIENCE

Data science is an interdisciplinary field that uses scientific methods, processes, algorithms and systems to extract knowledge and insights from structured and unstructured data, and apply knowledge and actionable insights from data across a broad range of application domains.

The term "data science" has been traced back to 1974, when Peter Naur proposed it as an alternative name for computer science. In 1996, the International Federation of Classification Societies became the first conference to specifically feature data science as a topic. However, the definition was still in flux.

The term “data science” was first coined in 2008 by D.J. Patil, and Jeff Hammerbacher, the pioneer leads of data and analytics efforts at LinkedIn and Facebook. In less than a decade, it has become one of the hottest and most trending professions in the market.

Data science is the field of study that combines domain expertise, programming skills, and knowledge of mathematics and statistics to extract meaningful insights from data.

Data science can be defined as a blend of mathematics, business acumen, tools, algorithms, and machine learning techniques, all of which help us in finding out the hidden insights or patterns from raw data, which can be of major use in the formation of big business decisions.

1.1.1 Data Scientist:

Data scientists examine which questions need answering and where to find the related data. They have business acumen and analytical skills as well as the ability to mine, clean, and present data. Businesses use data scientists to source, manage, and analyze large amounts of unstructured data.

Required Skills for a Data Scientist:

- Programming: Python, SQL, Scala, Java, R, MATLAB.

- Machine Learning: Natural Language Processing, Classification, Clustering.
- Data Visualization: Tableau, SAS, D3.js, Python, Java, R libraries.
- Big data platforms: MongoDB, Oracle, Microsoft Azure, Cloudera.

1.2 ARTIFICIAL INTELLIGENCE

Artificial intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. The term may also be applied to any machine that exhibits traits associated with a human mind such as learning and problem-solving.

Artificial intelligence (AI) is intelligence demonstrated by machines, as opposed to the natural intelligence displayed by humans or animals. Leading AI textbooks define the field as the study of "intelligent agents" any system that perceives its environment and takes actions that maximize its chance of achieving its goals. Some popular accounts use the term "artificial intelligence" to describe machines that mimic "cognitive" functions that humans associate with the human mind, such as "learning" and "problem solving", however this definition is rejected by major AI researchers.

Artificial intelligence is the simulation of human intelligence processes by machines, especially computer systems. Specific applications of AI include expert systems, natural language processing, speech recognition and machine vision.

AI programming focuses on three cognitive skills: learning, reasoning and self correction.

Learning processes: This aspect of AI programming focuses on acquiring data and creating rules for how to turn the data into actionable information. The rules, which are called algorithms, provide computing devices with step-by-step instructions for how to complete a specific task.

Reasoning processes: This aspect of AI programming focuses on choosing the right algorithm to reach a desired outcome.

Self-correction processes: This aspect of AI programming is designed to continually fine-tune algorithms and ensure they provide the most accurate results possible.

1.3 NATURAL LANGUAGE PROCESSING (NLP):

Natural language processing (NLP) allows machines to read and understand human language. A sufficiently powerful natural language processing system would enable natural-language user interfaces and the acquisition of knowledge directly from human-written sources, such as newswire texts. Some straightforward applications of natural language processing include information retrieval, text mining, question answering and machine translation. Many current approaches use word co-occurrence frequencies to construct syntactic representations of text. "Keyword spotting" strategies for search are popular and scalable but dumb; a search query for "dog" might only match documents with the literal word "dog" and miss a document with the word "poodle". "Lexical affinity" strategies use the occurrence of words such as "accident" to assess the sentiment of a document. Modern statistical NLP approaches can combine all these strategies as well as others, and often achieve acceptable accuracy at the page or paragraph level. Beyond semantic NLP, the ultimate goal of "narrative" NLP is to embody a full understanding of commonsense reasoning. By 2019, transformer based deep learning architectures could generate coherent text.

1.4 MACHINE LEARNING (ML) :

Machine learning is a way to predict what might happen in the future by looking at past data. It is a type of artificial intelligence that allows computers to learn on their own, without being programmed step by step. The main focus of machine learning is to create computer programs that can change and improve when they are given new data. It also covers the basics of machine learning and shows how to implement a simple machine learning algorithm using Python. The process of training and making predictions involves using special algorithms. These algorithms are given training data, and then they use that data to make predictions about new, unseen data. Machine learning can generally be divided into three main categories: supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, the program is given both input data and the correct answers, which are labeled by a human in advance. In unsupervised learning, the data is not labeled, and the algorithm must find patterns or group similar data on its own. Reinforcement learning is different because it involves an algorithm interacting with its environment and learning from the feedback it receives, either positive or negative, to improve how it performs.



Process of Machine Learning

Fig 1.1: Process of Machine Learning

1.5 OBJECTIVES:

This analysis focuses on identifying which features are most useful in predicting different types of network attacks, including DOS, R2L, U2R, Probe, and combinations of attacks. It also aims to uncover general trends that can assist in choosing the best model and tuning its hyperparameters. To do this, machine learning classification methods were used to build a model that can predict the discrete class of new input data.

The repository is a learning exercise meant to:

- Apply fundamental machine learning concepts using an available dataset, evaluate and interpret the results, and justify the interpretation based on the observed data.
- Create notebooks that act as computational records, documenting the thought process and investigating whether a network connection is under attack, in order to analyze the dataset.
- Evaluate and analyze statistical and visualized results to identify standard patterns across all categories.

1.6 PROJECT GOALS:

Exploration data analysis involves identifying variables

- Loading the dataset provided
- Importing necessary libraries and packages
- Examining the general characteristics of the data
- Finding duplicates and missing values
- Checking unique values and their counts

Exploration data analysis for single variable data

- Renaming, adding, and removing data as needed
- Specifying the data types

Exploration data analysis for two or more variables

- Creating visualizations such as pair plots, heatmaps, bar charts, and histograms

Method for detecting outliers and performing feature engineering

- Preprocessing the dataset
- Splitting the data into testing and training sets
- Comparing the Decision Tree, Logistic Regression, and Random Forest models

Comparing different algorithms to predict the outcome

- Based on which model provides the best accuracy

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

A literature review is a type of writing that looks at the important ideas and methods used in studying a specific topic. It uses existing research, or secondary sources, to discuss information in a certain subject area, often focusing on what has been written about it within a specific time frame. The main purpose is to help readers understand the latest research on a topic and can serve as a foundation for future research or as part of a research proposal. It can also be a simple summary of what has been written. Typically, it has a clear structure and includes both summaries and syntheses of the sources.

A summary is a brief overview of the key points from a source, while a synthesis is a more in-depth reorganization of the information.

It may offer a new way of looking at old ideas, combine old and new interpretations, or show how the field has developed over time, including major discussions and disagreements. In some cases, the literature review may assess the sources and suggest which ones are most relevant or useful.

Loan default patterns have been studied for a long time from a social and economic perspective.

Many economic studies use models based on real data to predict the likelihood of someone defaulting on a loan.

Recently, there's been an increasing interest in using machine learning techniques for these kinds of predictions. Some studies aim to understand both past and current views on whether loans are approved or not.

REVIEW OF LITERATURE SURVEY

Title: A Prediction Model of DoS Attack's Distribution Discrete Probability

Author: Wentao Zhao, Jianping Yin and Jun Long

Year: 2008

The process of prediction analysis involves using a method or technology to explore or simulate unknown or complex intermediate steps, based on previous and current states, to predict future outcomes. In an early warning system, accurately predicting DoS attacks is a key objective in network defense and attack tasks. Detection based on identifying abnormal behavior is effective for spotting DoS attacks. Many studies have examined DoS attacks from different angles. However, these methods often require prior knowledge and struggle to distinguish between normal traffic spikes and actual DoS attacks. Additionally, they need a large amount of historical data and are not efficient for predicting such attacks. Based on data from traffic monitoring and intrusion detection, this paper proposes a prediction model for the distribution of DoS attacks using a clustering method based on a genetic algorithm and the Bayesian method. The process starts with solving the clustering problem, then uses the genetic algorithm to optimize the clustering method. Using the optimized clustering on sample data, we identify various relationships between traffic and attack volumes, and then build several prediction sub-models for DoS attacks. Further, by applying the Bayesian method, we calculate the discrete probability for each sub-model and develop a distribution discrete probability prediction model for DoS attacks. This paper starts by examining the relationship between network traffic data and the amount of DoS attacks, and then proposes a clustering method based on a genetic optimization algorithm to classify DoS attack data. This method first identifies the appropriate partitioning of the relationship between network traffic and the amount of DoS attacks using optimized clustering and then builds the prediction sub-models for DoS attacks.

Title: Apriori Viterbi Model for Prior Detection of Socio-Technical Attacks Social Network

Author: Preetish Ranjan, Abhishek Vaish

Year: 2014

Socio-technical attacks are organized efforts where people misuse technology with harmful intentions to damage the social structure that relies on trust and faith. Harmful advertisements on the internet and mobile phones can damage individuals, organizations, groups, and brands, which can be very serious. People are very sensitive about

their religion, so spreading manipulated information against their beliefs can create chaos in society and might lead to social unrest, political instability, and other problems. Cyberattacks on essential systems like water, electricity, finance, healthcare, food, and transportation can cause widespread chaos in a short time. These attacks can be even more destructive than traditional bombs because they don't cause physical harm but attack the trust and faith that are the foundation of our social systems. Trust is the belief that someone will act in a way that meets expectations, and it starts at home and builds up to form a society. Trust in information can be built if it comes from a trustworthy source or is validated by an authentic authority, creating a sense of security and hope. In the large and complex social networks created through cyberspace and telecommunications, identifying or predicting socio-technical attacks is very challenging. This challenge offers an opportunity to explore various methods, ideas, and algorithms to detect such attacks based on their patterns, properties, structure, and trends. It aims to uncover hidden information in large social networks by compressing them into smaller networks using the apriori algorithm and then diagnosing them with the Viterbi algorithm to predict the most likely conversation patterns in the network. If these patterns match those of criminals, terrorists, or hijackers, it can help generate warnings before any crime occurs. The rise of the internet and mobile phones has made it easier for people to share information through social networking sites, blogs, reviews, ratings, bookmarking, social news, media sharing, and Wikipedia. Careful analysis of these patterns can reveal whether someone is communicating in a harmful or harmless way with another user, potentially leading to socio-technical attacks. From the simulation conducted on Call Detail Records (CDR), it can be concluded that if such simulations are applied to internet-based networks and we have access to data that can be transformed into transition and emission matrices, various predictions can be made. These predictions would be useful in making informed decisions.

Title: New Attack Scenario Prediction Methodology

Author: Seraj Fayyad, Cristoph Meinel

Year: 2013

Intrusion detection systems (IDS) are used to spot malicious activities happening on an IT system. These systems monitor and analyze what's happening within the IT system to identify any suspicious or harmful actions. In an ideal situation, the IDS generates an alert for every detected malicious activity and saves it in its database. Some of these alerts are connected. The way these alerts are related can be either duplication or part of the same attack scenario. Duplication means two alerts come from the same malicious activity. When alerts are part of the same attack scenario, it means they come from related malicious activities. An attack scenario, or a multi-step attack, is a series of connected malicious activities carried out by the same attacker to achieve a specific goal. The normal

relationship between these malicious activities is usually causal. Causal relationship means that the output of one malicious activity serves as the starting point for the next one. A multi-step attack against a network often starts with gathering information about the network, which is done through network reconnaissance and fingerprinting. During reconnaissance, network settings and running services are identified. Through fingerprinting, the type and version of the operating system are determined. We propose a real-time method to predict the most likely attack steps and scenarios. This method uses historical attack data and information from an attack graph. It doesn't require heavy computation, like checking a library of attack plans. It allows for parallel prediction of multiple attack scenarios. The next possible step could be identifying an attack plan based on the modeled attack graph from the previous step. An attack plan usually involves exploiting a sequence of discovered vulnerabilities. Often, this sequence is spread across multiple network nodes. These nodes are linked through causal relationships and network connectivity. Finally, the attacker systematically exploits the sequence of vulnerabilities in the attack scenario to reach their goal. An attack plan includes several connected malicious activities that ultimately result in achieving the attacker's objective.

Title: Cyber Attacks Prediction Model Based on Bayesian Network

Author: Jinyu W1, Lihua Yin and Yunchuan Guo

Year: 2012

The prediction results show the security status of the target network in the future, allowing security administrators to take necessary actions to improve network security based on these findings. To accurately predict potential attacks, attack probability is a key factor. It helps us understand the likelihood of intruders gaining access. As a vital part of network security evaluation, attack probability and its calculation methods have been studied for a long time. Many models have been developed for assessing network security. Among them, graphical models like attack graphs have become the popular choice. Attack graphs show how different vulnerabilities and exploits are connected, helping us see all the possible paths an attacker might use to break into the network. The traffic patterns to various hosts or servers can be different. Hosts or servers with higher traffic are often more important, and attackers may have more opportunities to interact with them. In our cyber-attack prediction model, we use attack graphs to identify vulnerabilities within the network. We also take into account three key environment factors that greatly influence future cyber-attacks: the value of assets in the network, how the network is used, and the history of past attacks. Cyber-attack prediction is an essential part of risk management. However, existing prediction methods often fail to consider the specific environment of the target network, which can lead to incorrect results. In this paper, we introduce a new cyber-attack prediction model based on a Bayesian network. We use attack

graphs to show all the vulnerabilities and possible attack paths. Then, we use a Bayesian network to capture the environment factors. Cyber-attack predictions are made using the constructed Bayesian network.

Title: A Prediction Model of DoS Attack's Distribution Discrete Probability

Author: Wentao Zhao, Jianping Yin

Year: 2008

This paper starts by exploring the connection between network traffic data and the level of DoS attack activity. It then introduces a clustering method that uses a genetic optimization algorithm to classify DoS attack data. The approach first establishes an accurate relationship between network traffic and DoS attack levels through optimized clustering, and then creates prediction sub-models for DoS attacks. Using the Bayesian method, the paper calculates the probability outputs for each sub-model, allowing for a prediction of the distribution of DoS attack levels in the future. The paper first outlines the clustering problem and then applies the genetic algorithm to refine clustering techniques. Based on this optimized clustering of sample data, various categories of the relationship between traffic and attack levels are identified, leading to the creation of multiple prediction sub-models for DoS attacks. Additionally, the Bayesian approach is used to calculate discrete probabilities for each sub-model, resulting in a discrete probability prediction model for DoS attacks.

Title: Adversarial Examples: Attacks and Defenses for Deep Learning

Author: Xiaoyong Yuan, Pan He, Qile Zhu, and Xiaolin Li

Year: 2019

It looked at the latest discoveries about adversarial examples in deep neural networks. We studied the current techniques used to create adversarial examples. A classification system for these examples was introduced. We also looked into how adversarial examples are used and the ways to protect against them. This paper aimed to cover the most up-to-date research on adversarial examples within the deep learning field. When compared to recent studies, we examined the current issues and possible fixes related to adversarial examples. However, deep neural networks have been found to be weak against specially crafted input samples known as adversarial examples. These perturbations are nearly invisible to humans but can trick DNNs during testing or when they are in use. The risk posed by adversarial examples has become a big concern when applying DNNs in important safety areas. Because of this, both attacking and defending against adversarial examples have become widely studied topics. In this paper, we review the recent developments on adversarial examples for DNNs, summarize

the methods used to generate them, and suggest a classification of these methods. Within this classification, we examine the applications of adversarial examples. We also go into detail about the strategies to counter them. Additionally, we discuss three main challenges in dealing with adversarial examples and the possible solutions to these challenges.

CHAPTER-3

AIM AND SCOPE OF THE PRESENT INVESTIGATION

3.1 PROJECT PROPOSAL

The idea is to create a machine learning model that can detect anomalies. Anomaly detection is a key method for spotting things like fraud, unusual activities, network attacks, and other irregular events that are important but hard to find. To build the model, we use data science techniques such as identifying the right variables, which include both the factors we're looking at and the ones that influence them. We also visualize the data to better understand the patterns and insights within it. The model is developed using past data, allowing the algorithm to learn from it. Different algorithms are tested to find the best one, and then we calculate and compare the performance metrics to evaluate how well each one works.

3.2 SCOPE OF THE PROJECT

The scope of this project is to look into a dataset that includes network connection attacks, specifically for the medical sector, using machine learning methods. The aim is to determine whether a network connection is under attack or not.

3.3 EXISTING SYSTEM

They suggested starting with a contrastive self-supervised approach to tackle anomaly detection in attributed networks. Their method, called CoLa, includes three main parts: contrastive instance pair sampling, a GNN-based contrastive learning model, and a multi-round sampling method for calculating anomaly scores. Their model focuses on understanding the connection between each node and its surrounding structure, and it uses an objective related to anomalies to train the contrastive learning model. We think this framework offers a new way to bring self-supervised and contrastive learning into more graph-based anomaly detection tasks. The scores generated through multiple rounds by the contrastive learning model are then used to assess how abnormal each node is, based on statistical analysis. The process has two stages: training and inference. During training, the contrastive

learning model is trained using sampled instance pairs without needing labeled data. Then, during inference, the model produces anomaly scores for each node.

3.3.1 Disadvantages:

1. The performance is not good and it get complicated for other networks.
2. The performance metrics like recall F1 score and comparison of machine learning algorithm is not done.

3.4 PREPARING DATASET:

This Dataset contains 3000 records of features. It is classified into 4 classes.

- DOS Attack
- R2L Attack
- U2R Attack
- Probe Attack

3.5 STSTEM STUDY

3.5.1 Classification of Attacks:

The dataset from KDD Cup99 includes normal traffic and 22 different types of attacks, with a total of 41 features. Every traffic pattern in the dataset is labeled as either 'normal' or one of the attack types for future analysis. Over time, various types of attacks enter the network, and these attacks are grouped into four main categories.

These categories are:

- Denial of Service (DoS)
- User to Root (U2R)
- Remote to User (R2L)
- Probing

DENIAL OF SERVICE (DoS) :

Denial of Service is a type of attack where an attacker overwhelms a computer or its memory, making it too busy or full to handle real requests. This stops legitimate users from using the machine. There are several ways to carry out a DoS attack, such as by misusing the computer's normal features, targeting flaws in how the system is built,

or using incorrect settings in the system. DoS attacks are grouped based on which services they make unavailable to real users.

User to Root (U2R) :

In User to Root attack, an attacker starts with access to a normal user account on the system and gains root access. Regular programming mistakes and environment assumption give an attacker the opportunity to exploit the vulnerability of root access.

REMOTE TO USER (R2U) :

In Remote to User attack, an attacker sends packets to a machine over a network that exploits the machine's vulnerability to gain local access as a user illegally. There are different types of R2L attacks and the most common attack in this class is done by using social engineering.

PROBING:

Probing is a class of attacks where an attacker scans a network to gather information in order to find known vulnerabilities. An attacker with a map of machines and services that are available on a network can manipulate the information to look for exploits. There are different types of probes: some of them abuse the computer's legitimate features and some of them use social engineering techniques. This class of attacks is the most common because it requires very little technical expertise

3.5.2 ATTACKS SUMMARY:

This chapter outlines the structure of the dataset used in the proposed work. The various kinds of features such as discrete and continuous features are studied with a focus on their role in the attack. The attacks are classified with a brief introduction to each. The next chapter discusses the clustering and classification of the data with a direction to learning by machine.

Dos	R2L	U2R	Probe
Back	FTP Write	Load module	Ip sweep
Neptune	Multihop	Rerl	Nmap
Land Pod	Phf	Rootkit	Satan

Smurf	Spy	Buffer overflow	Port sweep
Teardrop	Warezcclient	Ps	Msscan saint
Apache2	Warezmaster	Sql attack xterm	
Mail bomb	Imap		
Process table	Guess password		
UDP Storm	http tunnel		
	named		
	send mail		
	snmpget attack		
	snmp guess		
	worm		
	xlock		
	xsnoop		

Table 3.1: Attack Types Grouped to respective Class

Types of Attacks	Description
back	Denial of service attack against apache web server where a client requests a URL containing many backslashes
neptune	Syn flood denial of service on one or more ports
land	Denial of service where a remote host is sent a UDP packet with the same source and destination
pod	Denial of service ping of death
smurf	Denial of service icmp echo reply flood

teardrop	Denial of service where mis-fragmented UDP packets cause some systems to reboot
multihop	Multi-day scenario in which a user first breaks into one machine
phf	Exploitable CGI script which allows a client to execute arbitrary commands on a machine with a mis-configured web server.
spy	Multi-day scenario in which a user breaks into a machine with the purpose of finding important information where the user tries to avoid detection. Uses several different exploit methods to gain access
warezclient	Users downloading illegal software which was previously posted via anonymous FTP by the warezmaster
warezmaster	Anonymous FTP upload of Warez (usually illegal copies of copy wrieded software) onto FTP server
Imap	Remote buffer overflow using imap port leads to root shell
loadmodule	Non-stealthy loadmodule attack which resets IFS for a normal user and creates a root shell
Perl	Perl attack which sets the user id to root in a perl script and creates a root shell
rootkit	Multi-day scenario where a user installs one or more components of a rootkit
ipsweep	Surveillance sweep performing either a port sweep or ping on multiple host addresses
nmap	Network mapping using the nmap tool. Mode of exploring network will vary-options include SYN

satan	Network probing tool which looks for well-known weaknesses. Operates at three different levels. Level 0 is light
portsweep	Surveillance sweep through many ports to determine which services are supported on a single host
dict	Guess passwords for a valid user using simple variants of the account name over a telnet connection
eject	Buffer overflow using eject program on Solaris. Leads to a user->root transition if successful
ffb	Buffer overflow using the ffbconfig UNIX system command leads to root shell
format	Buffer overflow using the fdformat UNIX system command leads to root shell
ftp-write	Remote FTP user creates .rhost file in world writable anonymous FTP directory and obtains local login
guest	Try to guess password via telnet for guest account
syslog	Denial of service for the syslog service connects to port 514 with unresolvable source ip
warez	User logs into anonymous FTP site and creates a hidden directory

Table 3.2: Description of Attacks

3.6 FEASIBILITY STUDY

3.6.1 Data wrangling:

In this section of the report will load in the data, check for cleanliness, and then trim and clean given dataset for analysis. Make sure that the document steps carefully and justify for cleaning decisions.

3.6.2 Data Collection:

The data set collected for predicting given data is split into Training set and Test set. Generally, 7:3 ratios are applied to split the Training set and Test set. The Data Model which was created using Random Forest, logistic, Decision tree algorithms and Support vector classifier (SVC) are applied on the Training set and based on the test result accuracy, Test set prediction is done.

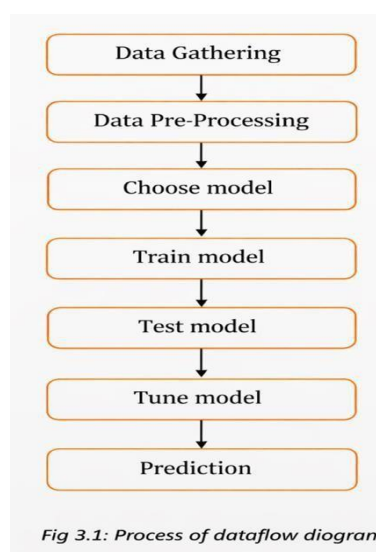
3.6.3 Preprocessing:

The data that was collected could have missing values, which might cause inconsistencies. To get better results, the data needs to be preprocessed to make the algorithm work more efficiently. Outliers should be removed, and the variables need to be converted properly.

3.6.4 Building the classification model:

The prediction of phishing websites using a Random Forest algorithm works well for several reasons. It performs better in classification tasks. It handles preprocessing well, especially with outliers, irrelevant data, and different types of variables like continuous, categorical, and discrete. It also gives an out-of-bag error estimate, which has been shown to be unbiased in many cases and is easier to adjust.

3.6.5 Construction of a Predictive Model:



CHAPTER-4

EXPERIMENTAL OR MATERIALS AND METHODS; ALGORITHMS USED

4.1 PROJECT REQUIREMENTS:

Requirements are the basic constraints that are required to develop a system. Requirements are collected while designing the system. The following are the requirements that are to be discussed.

1. Functional requirements
2. Non-Functional requirements
3. Environment requirements
 - A. Hardware requirements
 - B. software requirements

4.1.1 Functional Requirements:

The software requirements specification is a technical document that outlines the needs for a software product. It is the first stage in analyzing requirements. It includes the specific needs of a particular software system. The following details should follow the use of special libraries such as SK-learn, pandas, NumPy, matplotlib, and seaborn.

4.1.2 Non-Functional Requirements:

Process of functional steps,

1. Problem defines
2. Preparing data
3. Evaluating algorithms
4. Improving results
5. Prediction the result

4.1.3 Environment Requirement:

1. Software Requirements:

Operating System : windows 11 Tool:

Anaconda with Jupyter Notebook

2. Hardware Required

Processor : Pentium IV/III Hard Disk :

minimum 90 GB RAM : minimum 4

GB

4.2 Software Description :

Anaconda is a free and open-source platform that provides the Python and R programming languages for use in scientific computing, such as data science, machine learning, handling large datasets, and predictive analytics. Its main goal is to make it easier to manage and deploy packages. It uses a package management system called Conda to handle different versions of packages. Anaconda is used by more than 12 million people and includes over 1,400 commonly used data science packages that work on Windows, Linux, and macOS. The Anaconda distribution also includes a tool called Anaconda Navigator, which helps manage packages and virtual environments. This means users don't have to install each library separately.

4.2.1 Anaconda Navigator:

Anaconda Navigator is a desktop graphical user interface (GUI) that comes with the Anaconda distribution. It lets you start applications and manage conda packages, environments, and channels without using command-line commands. With Navigator, you can search for packages on Anaconda.org or in a local Anaconda repository. If you're mainly working on data science projects, Anaconda is a great choice. It was created by Continuum Analytics and is a Python distribution that already includes many useful libraries for data science. Anaconda provides a version of Python and R for scientific computing, including data science, machine learning, large-scale data processing, and predictive analytics. Its main goal is to make it easier to manage packages and deploy them. Many scientific packages rely on specific versions of other packages, so data scientists often use multiple versions of different packages and separate them into different environments. The conda command-line tool acts as both a package manager and an environment manager, helping data scientists ensure that each package version has the correct dependencies and works properly. Navigator offers a simple, point-and-click way to handle packages and environments without needing to type conda commands in a terminal. You can use it to find, install, run, and update packages—all within Navigator.

The following apps come pre-installed in Navigator:

- JupyterLab
- Jupyter Notebook
- Spyder
- PyCharm
- VSCode
- Glueviz
- Orange 3 App
- RStudio
- Anaconda Prompt (only for Windows)
- Anaconda PowerShell (only for Windows)

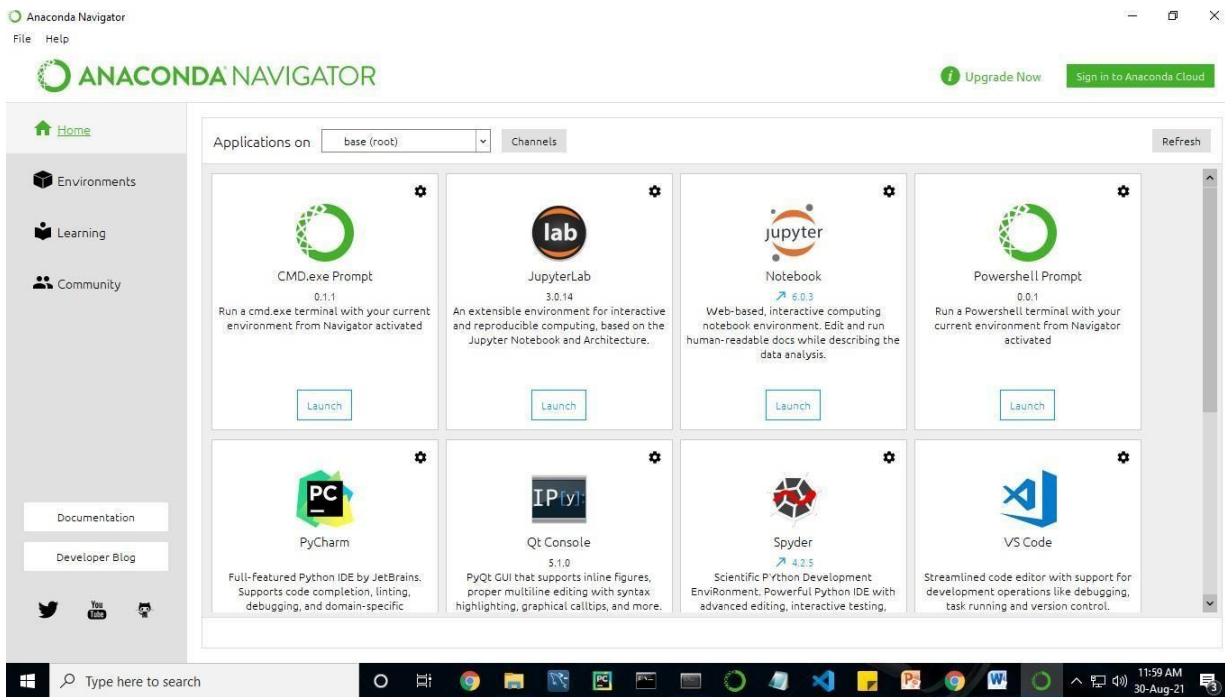


Fig 4.1: Anaconda Navigator Interface

4.2.2 Jupyter Notebook:

This website serves as a "meta" guide for the Jupyter ecosystem. It gathers various resources to help you explore the tools and communities within this ecosystem, as well as get started with using them. Project Jupyter is a project and a community that aims to "develop open-source software, open-standards, and services for interactive computing across dozens of programming languages." It started as a spin-off from IPython in 2014 by Fernando Perez.

Notebook documents are files created by the Jupyter Notebook application. These documents include both computer code, like Python, and rich text elements such as paragraphs, equations, figures, and links. Notebook documents are both readable by humans and contain the explanation of the analysis along with its results, such as figures and tables. They are also executable, meaning you can run them to perform data analysis.

Installation: The simplest way to install the Jupyter Notebook App is by installing a scientific Python distribution, which also comes with a variety of scientific Python packages. The most widely used distribution is called Anaconda.

Running the Jupyter Notebook:

To start the Jupyter Notebook app, you can click on the Jupyter Notebook icon that was installed with Anaconda in the start menu on Windows, or you can type "jupyter notebook" into a terminal on Windows.

This will open a new browser window or tab showing the Notebook Dashboard, which acts like a control panel allowing you to select which notebook to open.

Once it's running, the Jupyter Notebook app can only access files within its startup folder, including any subfolders.

If you place your notebooks in your home folder or its subfolders, no configuration is needed. However, if you're using a different folder, you'll need to set the startup folder so that it contains all your notebooks.

Saving notebooks: Changes to the notebooks are automatically saved every few minutes. To prevent modifying the original file, make a copy of the notebook by going to the File menu and selecting "Make a copy..." and then save your changes to the copied version.

Executing a notebook :

To run a notebook, first download the notebook file you want to use and place it in your notebook folder, or in a subfolder if needed.

Launch the Jupyter Notebook application.

Once the Notebook Dashboard appears, locate the notebook you've placed there. Clicking on its name will open it in a new browser tab.

To get familiar with the interface, click on the menu option Help and then User Interface Tour.

To execute the notebook step by step, press shift + enter.

If you want to run all the cells at once, go to the Cell menu and select Run All.

If you need to reset the computing environment, go to the Kernel menu and choose Restart. This is helpful when you want to start fresh, as it clears all variables and closes any open files.

Purpose: To help with interactive data science and scientific computing in all programming languages.

File Extension: An IPYNB file is a notebook created by Jupyter Notebook, which is an interactive tool that allows scientists to work with and analyze data using Python.

4.2.3 PYTHON:

Introduction :

Python is an interpreted, high-level, general-purpose programming language. It was designed with a focus on making code easy to read, and it uses significant indentation to define blocks of code. The language's structure and object-oriented features are meant to help programmers write clear and logical code, whether for small or large projects. Python is dynamically typed, meaning you don't have to declare variable types explicitly, and it uses automatic garbage collection to manage memory. It

supports multiple programming styles, including structured (especially procedural), object-oriented, and functional programming. Python is often called a "batteries included" language because it comes with a comprehensive standard library that includes many built-in modules and functions. Guido van Rossum started working on Python in the late 1980s as a follow-up to the ABC programming language. The first version, Python 0.9.0, was released in 1991. Python 2.0 came out in 2000 and introduced features like list comprehensions and a garbage collection system based on reference counting. Python 3.0 was released in 2008 and was a major update that isn't fully backward compatible with Python 2. Python 2 was officially discontinued with the last release, version 2.7.18, in 2020. Python is consistently one of the most popular programming languages in the world.

History:

Python was created in the late 1980s by Guido van Rossum at Centrum Wiskunde & Informatica, also known as CWI, in the Netherlands. It was designed as a replacement for the ABC programming language, which was influenced by SETL. ABC had features like exception handling and could work with the Amoeba operating system. The actual development of Python began in December 1989. Van Rossum was the main person in charge of the project, acting as the lead developer. He continued in this role until July 12, 2018, when he announced he was taking a "permanent vacation" from his duties. This title, "Benevolent Dictator for Life," was given to him by the Python community to show how much he had contributed and led the project over the years. In January 2019, active Python core developers chose a group of five people to take over leadership, forming a "Steering Council." As of 2021, the current members of this council are Barry Warsaw, Brett Cannon, Carol Willing, Thomas Wouters, and Pablo Galindo Salgado.

Python 2.0 came out on October 16, 2000, and brought several big changes, such as a garbage collector that could detect cycles and support for Unicode. Python 3.0 was released on December 3, 2008. This was a big update that wasn't fully backward-compatible with earlier versions. Many important features from Python 3 were later added to Python 2.6.x and 2.7.x. Python 3 releases also included tools called "2 to 3 utilities," which help with converting Python 2 code to Python 3, at least partially.

Python 2.7's end-of-life was first scheduled for 2015, but it was moved to 2020 because many existing programs had trouble switching to Python 3.

After that date, there will be no more security updates or improvements for Python 2. With Python 2 reaching its end of life, only Python 3.6 and later versions are now supported. Python 3.9.2 and 3.8.8 were released quickly because all versions of Python, including 2.7, had security issues. These problems could allow for remote code execution and web cache poisoning.

Design Philosophy & Feature:

Python is a programming language that supports multiple programming styles. It fully supports object-oriented programming and structured programming, and it also has features that make it suitable for functional programming and aspect-oriented programming, including through the use of meta-programming and meta-objects (also known as magic methods).

Python can also support other programming styles through extensions, such as design by contract and logic programming.

Python uses dynamic typing, which means that the type of a variable is determined at runtime.

It manages memory using a combination of reference counting and a garbage collector that can detect and clean up cycles. It also uses dynamic name resolution, which means that method and variable names are resolved during program execution, not at the time the code is written.

Python's design includes some support for functional programming, similar to the style used in the Lisp programming language.

It includes functions like filter, map, and reduce, as well as features like list comprehensions, dictionaries, sets, and generator expressions. The standard library also includes two modules—`itertools` and `functools`—that provide functional tools inspired by Haskell and Standard ML.

The core philosophy of Python is explained in a document called "The Zen of Python" (PEP 20), which includes sayings like:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Readability counts.

Rather than including all its functionality directly in the core language, Python was designed to be highly extensible through the use of modules.

This compact and modular structure has made it especially popular for adding programmable features to existing applications. Guido van Rossum's idea of a small core language combined with a large standard library and an easily extensible interpreter came from his frustrations with the ABC programming language, which took the opposite approach.

4.3 PYTHON LIBRARIES NEEDED

There are many libraries in python. in those we only use few main libraries needed.

4.3.1 NUMPY LIBRARY:

NumPy is an open-source library for Python that deals with numbers. It provides tools to work with multi-dimensional arrays and matrices. With NumPy, you can do many math tasks on these arrays, such as calculating trigonometric functions, statistics, and algebra, including things like average, mode, and standard deviation.

Installation: [<https://numpy.org/install/>]

[`pip install NUMPY`]

In this case, we mainly use arrays to calculate the mean and standard deviation.

4.3.3 MATPLOTLIB LIBRARY:

Matplotlib is a full-featured library that helps you create different types of visualizations in Python, such as static images, animated graphs, and interactive plots. It simplifies the process of making things easy and also allows you to tackle more complex tasks. You can use interactive figures that let you zoom in, move around, update data, and view information more clearly.

Installation guide: [<https://matplotlib.org/users/installing.html>] [PIP

INSTALL Matplotlib]

In this context, we mainly use pyplot for drawing graphs.

Matplotlib.pyplot is a set of functions that make the library work similarly to MATLAB. Each function in pyplot modifies a figure in some way, such as creating a new figure, adding a plotting area, drawing lines, and adding labels or other decorations to the graph.

4.3.4 SCIKIT-LEARN LIBRARY:

Scikit-learn is a free machine learning library for Python. It includes many algorithms such as support vector machines, random forests, regression, and k-neighbors. It also works well with other Python libraries like NumPy and SciPy.

Installation: <https://scikit-learn.org/stable/install.html>

[`pip install Scikit-Learn`]

Here, we use scikit-learn's regression methods for making predictions.

4.3.5 TKINTER :

Standard python interface to the GUI toolkit

Accessible to everybody and reusable in various contexts.

4.4 SYSTEM ARCHITECTURE

Development of different parts of a system. Software with effective modularity is easier to develop because functions can be separated into different sections, making it simpler to manage and understand. Software architecture is based on modularity, which means the software is divided into separate, named, and identifiable parts called modules. These modules are put together to meet the needs of the problem being solved. Modularity is the key quality that makes a program easier to think about and work with. There are five main factors that help us assess how well a design method creates an effective modular structure: modular decomposability, modular composability, modular understandability, modular continuity, and modular protection.

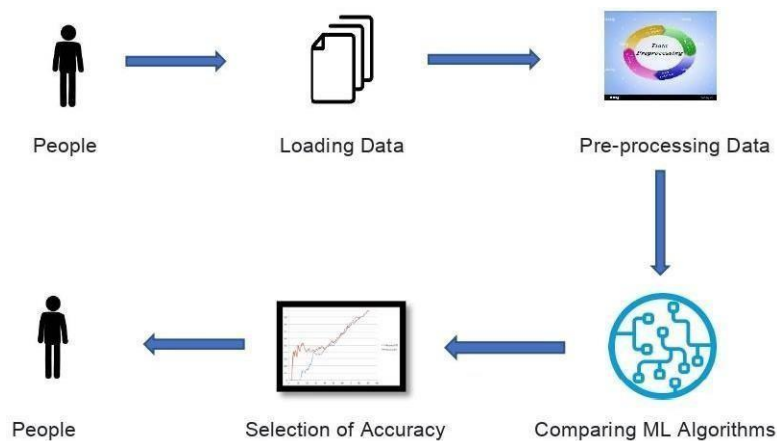


Fig 4.2: System Architecture

4.5 UML DIAGRAMS

4.5.1 Class Diagram

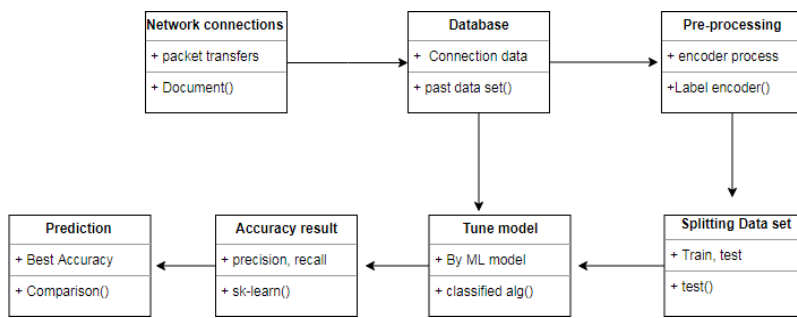


Fig 4.3: Class Diagram

A class diagram is a visual way to show the static part of a system and covers different parts of the application. So, putting together several class diagrams gives a full picture of the system. The name of each class diagram should be clear and explain what part of the system it shows. Before drawing, you should figure out each element and how they connect to each other. You also need to clearly define what each class does, including its properties and actions. It's best to keep the number of properties as small as possible because too many can make the diagram hard to understand. Use notes when needed to explain parts of the diagram, so that the developer or programmer can understand it easily. Finally, before finalizing the diagram, draw it on plain paper and revise it several times to make sure it's accurate and clear.

4.5.2 Use Case Diagram

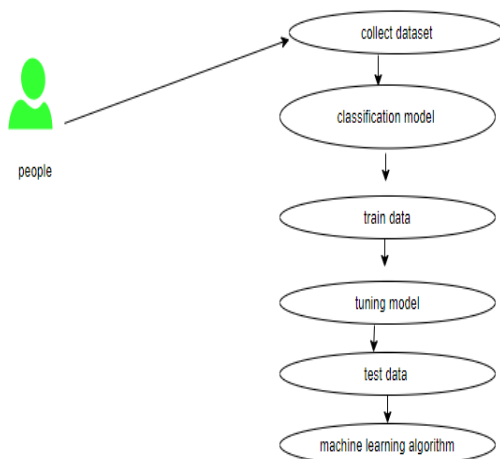
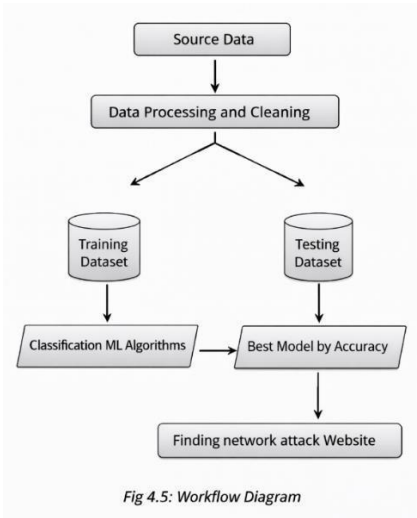


Fig 4.4: Use Case Diagram

Use case diagrams are used for analyzing the high-level requirements of a system. When analyzing the requirements, the different functionalities of the system are represented through use cases. So, in simple terms, use cases are just the system's functionalities written in a structured and organized way.

4.5.3 Workflow Diagram:



4.5.4 ACTIVITY DIAGRAM

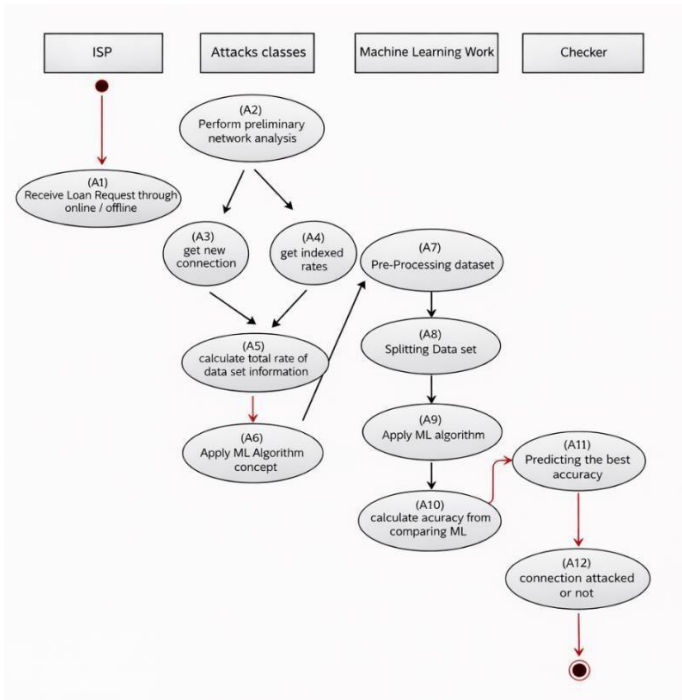


Fig 4.6: Activity Diagram

Activity is a specific action performed by a system. Activity diagrams are used to show how a system works over time, and they can also help build an actual working system through forward and reverse engineering. The only part missing in an activity diagram is the message exchange between activities. It doesn't display any messages sent from one activity to another. Sometimes, activity diagrams are thought of as flowcharts. Although they may look similar to

flowcharts, they are not exactly the same. They can show different types of flows, such as parallel, branched, concurrent, and single flows.

4.5.5 SEQUENCE DIAGRAM

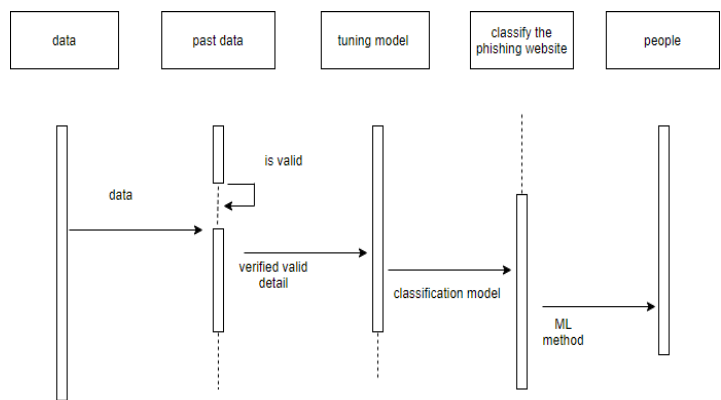


Fig 4.7: Sequence Diagram

Sequence diagrams visually show how logic flows through your system, helping you to document and check your logic. They are widely used for both analyzing and designing systems. Among UML tools, sequence diagrams are the most commonly used for dynamic modeling, which focuses on understanding how the system behaves. Other dynamic modeling methods include activity diagrams, communication diagrams, timing diagrams, and interaction overview diagrams. In my view, sequence diagrams, along with class diagrams and physical data models, are the most important models to use when designing modern business applications.

4.5.6 ENTITY RELATION DIAGRAM

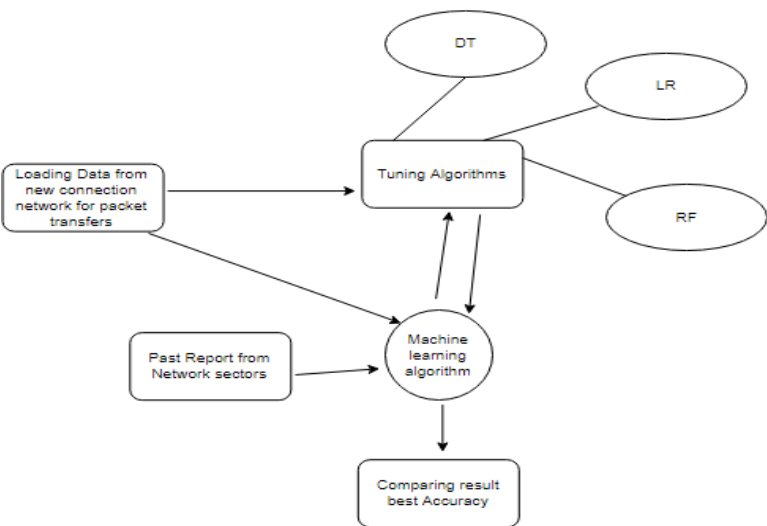


Fig 4.8: Entity Relationship Diagram (ERD)

An entity relationship diagram, or ERD, is a visual tool that shows how different parts of an information system connect. It represents the relationships between people, things, locations, ideas, or events in a system. This type of diagram is a way to model data and can help explain how business processes work. It's often used as a starting point for designing a relational database and can also help identify what information an organization needs. Even after a database is set up, an ERD can be useful for troubleshooting or improving business processes later on.

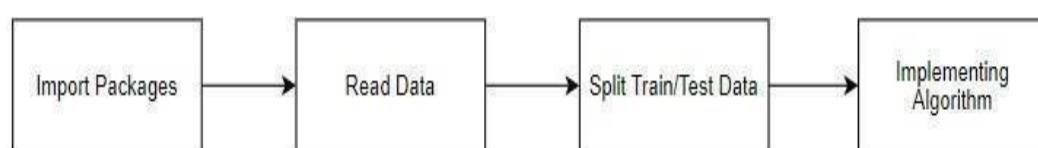
4.6 ALGORITHM USED

Logistic Regression :

Logistic regression is a statistical technique used to analyze a dataset where there are one or more variables that influence an outcome. The outcome is measured using a variable that has only two possible results, like yes or no. The main aim of logistic regression is to create the best model that explains how the binary outcome relates to a group of other variables. It is a classification method in machine learning that predicts the likelihood of a categorical result. In this method, the outcome variable is binary, meaning it can only take on two values, typically represented as 1 for success or yes, and 0 for failure or no.

In simpler terms, the logistic regression model predicts the probability that the outcome Y is 1, based on the values of X. Here are the key assumptions for logistic regression:

- The dependent variable must be binary, meaning it has only two possible outcomes.
- The category labeled as 1 in the dependent variable should be the one you're interested in predicting.
- Only the variables that are relevant to the outcome should be included in the model.
- The independent variables should not be highly correlated with each other, as this can cause problems in the model.
- The relationship between the independent variables and the log odds of the outcome should be linear.
- Logistic regression typically requires a large number of observations to produce reliable results.



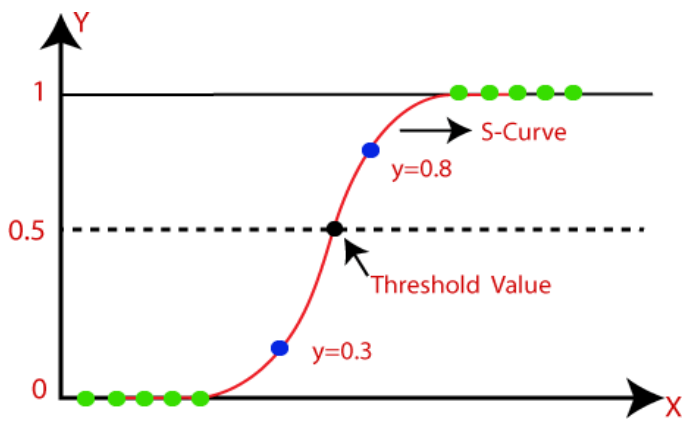


Fig 4.9 Logistic Regression

Decision Tree Classifier:

It is one of the most powerful and widely used algorithms. The decision tree algorithm is a type of supervised learning algorithm. It can work with both continuous and categorical types of output variables. Assumptions of a decision tree include:

- Initially, the entire training data is considered as the root.
- For calculating information gain, attributes are assumed to be categorical, while for other purposes, they are assumed to be continuous.
- Based on the attribute values, records are split into smaller groups recursively.
- Statistical methods are used to determine the order of attributes as the root or internal nodes.

A decision tree creates classification or regression models in the form of a tree structure. It splits a dataset into smaller and smaller subsets as the decision tree is built step by step. A decision node has two or more branches, and a leaf node represents a classification or final decision. The topmost node in the tree, which is the best predictor, is called the root node. Decision trees can handle both categorical and numerical data. They use an if-then rule set that is mutually exclusive and covers all possibilities for classification. These rules are developed one at a time using the training data. Once a rule is created, the data instances covered by that rule are removed. This process continues until a stopping condition is met. The tree is built from the top down using a divide-and-conquer approach. All attributes should be categorical, or they need to be converted into categorical form beforehand. Attributes at the top of the tree have a larger influence on the classification and are selected based on the concept of information gain. A decision tree can become too complex, creating too many branches, which may lead to overfitting and reflect errors due to noise or outliers.

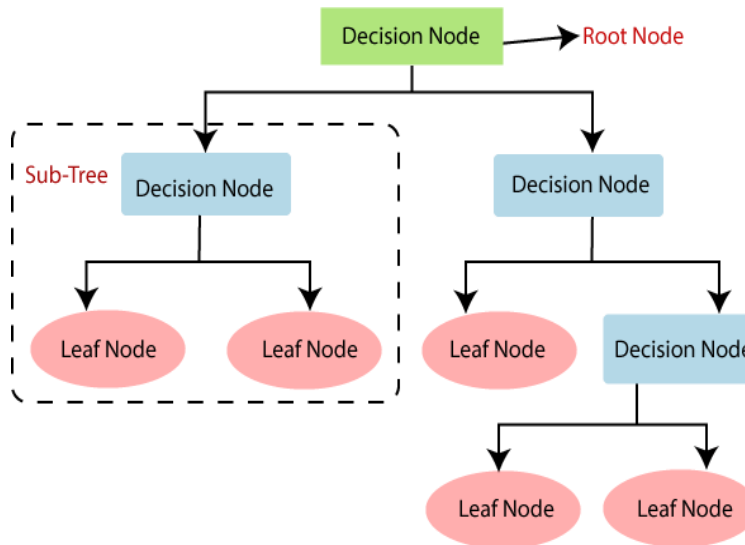


Fig 4.10: Decision Tree Classifier

Random Forest Classifier:

Random forests, also known as random decision forests, are an ensemble learning method used for classification, regression, and other tasks. They work by creating many decision trees during training and then using the most common class prediction (for classification) or the average prediction (for regression) from all the trees to make the final decision. Random forests help reduce the tendency of decision trees to overfit the training data. It is a supervised machine learning algorithm that falls under the category of ensemble learning.

Ensemble learning involves combining multiple algorithms, either of the same type or different types, to create a stronger and more accurate prediction model. In the case of random forests, multiple decision trees are combined, forming a "forest" of trees, which gives the method its name. Random forests can be used for both classification and regression tasks. Here are the basic steps for implementing the random forest algorithm:

- Select N random data points from the dataset.
- Create a decision tree using these selected data points.
- Decide how many trees you want in your model and repeat the above steps.

Random Forest Classifier

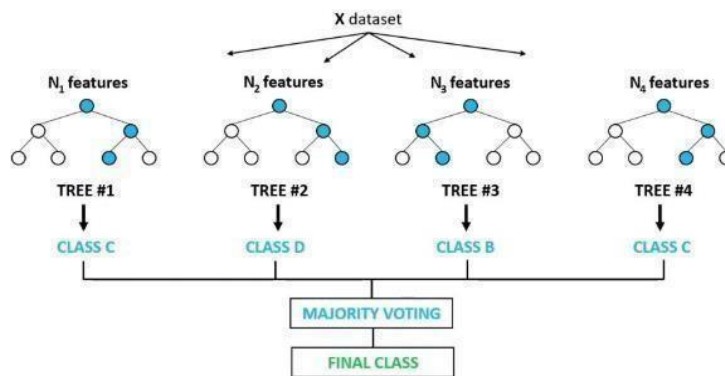


Fig 4.11: Random Forest Classifier

Support Vector Machine:

Support Vector Classifier, or SVC, is one of the most widely used supervised learning algorithms. It is used for both classification and regression tasks. However, it is mainly used for classification in machine learning. The purpose of the SVC algorithm is to find the best line or decision boundary that separates data in n -dimensional space into different classes. This helps in placing new data points into the correct category in the future. This best decision boundary is known as a hyperplane. The SVC algorithm selects the extreme points or vectors that help in forming this hyperplane. These extreme points are called support vectors, which is why the algorithm is named Support Vector Machine.

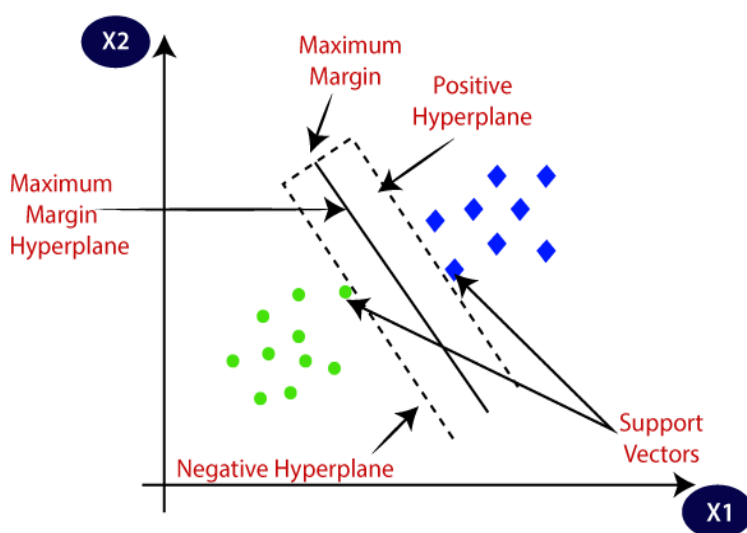


Fig 4.12: Support Vector Classifier

Graphical Neural Network (GNN):

A graph neural network (GNN) is a type of neural network designed to work with data that is structured as graphs. They gained popularity because of their use in supervised learning tasks related to the properties of different molecules.

Since their introduction, many variations of the message passing neural network (MPNN) framework have been developed.

These models improve the efficiency of GNNs for handling large graphs and have been applied in areas like social networks, citation networks, and online communities.

GNNs are a weaker version of the Weisfeiler–Lehman graph isomorphism test, meaning any GNN model is at least as effective as this test.

Researchers are working to combine GNNs with other geometric deep learning models in order to better understand how and why these models function.

In cases where the graph structure is not known, such as when building a k-nearest neighbor graph, it can be created using heuristic methods.

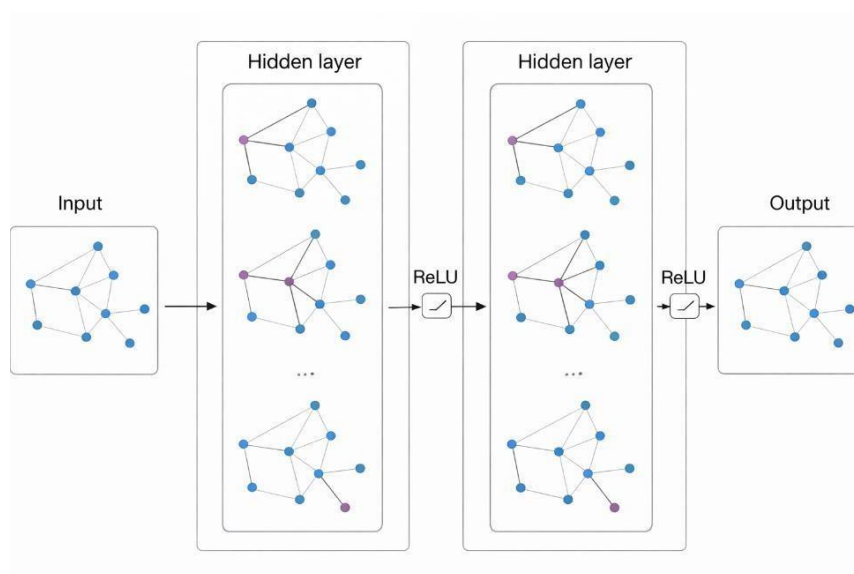


Fig : Graphical Neural Network

4.7 COMPARING ALGORITHM WITH PREDICTION IN THE FORM OF BEST ACCURACY RESULT

It is important to compare the performance of various machine learning algorithms consistently. You will learn how to create a test harness in Python using scikit-learn to compare multiple different machine learning algorithms. This test harness can be used as a template for your own machine learning problems, and you can add more algorithms to compare. Each model has different performance characteristics. By using resampling methods like cross-validation, you can get an estimate of how accurate each model may be on new, unseen data. You need to use these estimates to select one or two best models from the set of models you have developed. When you have a new dataset, it is a good idea to visualize the data using different techniques to look at it from different perspectives. The same idea applies to model selection. You should use various methods to examine the estimated accuracy of your machine learning algorithms in order to choose the one or two to finalize. One way to do this is by using different visualization methods to show the average accuracy, variance, and other properties of the distribution of model accuracies. In the next section, you will learn exactly how to do this in Python with scikit-learn. The key to a fair comparison of machine learning algorithms is ensuring that each algorithm is evaluated in the same way on the same data, which can be achieved by forcing each algorithm to be evaluated on a consistent test harness.

Pre-processing is the process of making changes to data before it is given to a machine learning algorithm. It is a method used to turn raw data into a clean and ready-to-use dataset. When data is collected from various sources, it is usually in an unorganized or raw format, which isn't suitable for analysis. To get better results from a machine learning model, the data must be prepared properly. Some machine learning models require data to be in a specific format. For instance, the Random Forest algorithm does not work with missing or null values. So, before applying this algorithm, we need to handle those missing values in the raw dataset. Another important point is that the data should be structured in a way that allows multiple machine learning and deep learning algorithms to be applied to the same dataset.

In the example below, five different algorithms are compared:

- Logistic Regression
- Random Forest
- Decision Tree Classifier
- Support Vector Classifier
- Graphical Neural Network

The K-fold cross validation process is used to evaluate each algorithm. It is important that all algorithms are configured with the same random seed to make sure the same splits of the training data are done each time, and that every algorithm is tested in exactly the same way. Before comparing the algorithms, a machine learning model is built using the Scikit-Learn library. In this library,

preprocessing is done, and different models are created including a linear model using the logistic regression method, cross validation using the K-Fold method, an ensemble model using the random forest method, and a tree-based model using the decision tree classifier. Additionally, the training set is split into a test set. The final results are predicted by comparing the accuracy of each model.

False Positives (FP): A person who will pay predicted as defaulter. When actual class is no and predicted class is yes. E.g., if actual class says this passenger did not survive but predicted class tells you that this passenger will survive.

False Negatives (FN): A person who default predicted as payer. When actual class is yes but predicted class in no. E.g., if actual class value indicates that this passenger survived and predicted class tells you that passenger will die.

True Positives (TP): A person who will not pay predicted as defaulter. These are the correctly predicted positive values which means that the value of actual class is yes and the value of predicted class is also yes. E.g., if actual class value indicates that this passenger survived and predicted class tells you the same thing.

True Negatives (TN): A person who default predicted as payer. These are the correctly predicted negative values which means that the value of actual class is no and value of predicted class is also no. E.g., if actual class says this passenger did not survive and predicted class tells you the same thing.

Prediction result by accuracy:

Logistic regression algorithm also uses a linear equation with independent predictors to predict a value. The predicted value can be anywhere between negative infinity to positive infinity. It needs the output of the algorithm to be classified variable data. Higher accuracy predicting result is logistic regression model by comparing the best accuracy.

True Positive Rate (TPR) = $TP / (TP + FN)$

False Positive Rate (FPR) = $FP / (FP + TN)$

Accuracy: The Proportion of the total number of predictions that is correct otherwise 46 overall how often the model predicts correctly defaulters and non- defaulters.

Accuracy calculation:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

Accuracy is the most straightforward way to measure how well a model performs. It's simply the number of correct predictions divided by the total number of predictions. Some people might believe that a high accuracy means the model is excellent. While accuracy is a useful metric, it's only reliable when the dataset is balanced, meaning the number of false positives and false negatives is about the same.

Precision:

$$\text{Precision} = TP / (TP + FP)$$

Precision is the percentage of positive predictions that are actually correct. It is calculated as TP divided by the sum of TP and FP. This metric tells us, out of all the passengers we predicted survived, how many really did survive. A high precision means a low rate of false positives. We have a precision of 0.788, which is quite good.

Recall:

The proportion of positive observed values correctly predicted. (The proportion of actual defaulters that the model will correctly predict) $\text{Recall} = TP / (TP + FN)$

Recall (Sensitivity) - Recall is the ratio of correctly predicted positive observations to the all observations in actual class - yes.

F1 score:

The F1 score is an average of Precision and Recall. This means it considers both false positives and false negatives. It's not as straightforward to understand as accuracy, but it's often more helpful, especially when the classes are not balanced. Accuracy is best when the costs of false positives and false negatives are similar. However, if these costs are very different, it's better to look at Precision and Recall individually.

$$\text{General Formula: } F\text{-Measure} = 2TP / (2TP + FP + FN)$$

$$\text{F1-Score Formula: } F1 \text{ Score} = 2 * (\text{Recall} * \text{Precision}) / (\text{Recall} + \text{Precision})$$

4.8 MODULES DESCRIPTION

1. Data validation process and Visualization
2. Prediction of DoS attacks
3. Prediction of R2L attacks
4. Prediction of U2R attacks
5. Prediction of Probe attacks
6. Prediction of overall network attacks
7. GUI based prediction results of Network attacks

4.8.1 Data validation process and Visualization (Module-01):

Validation techniques in machine learning help determine the error rate of a Machine Learning (ML) model, which can closely reflect the actual error rate of the dataset. If the data volume is large and representative of the entire population, you might not need these validation methods. However, in real-world situations, the data samples might not accurately represent the full dataset. It is important to identify missing values, duplicates, and the data type, such as whether a variable is a float or integer. A sample of data is used to fairly evaluate how well a model performs on the training dataset while adjusting its hyperparameters. As the model's performance on the validation dataset is used to fine-tune its settings, the evaluation becomes more biased. The validation set is used to evaluate a model, but this is done frequently. Machine learning engineers use this data to make small adjustments to the model's hyperparameters. Data collection, analysis, and the process of handling data content, quality, and structure can be time-consuming. During data identification, it helps to understand your data and its characteristics, which in turn helps you choose the right algorithm for building your model. For example, time series data can be analyzed using regression algorithms, while classification algorithms are suitable for discrete data. (For example, to show the format of data types in the given dataset.)

duration	protocol_type	service	flag	src_bytes	dst_bytes	land	Wrong_fragment	Urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate
0	0	udp	private	SF	105	146	0	0	0	0 ...	254	1.00
1	0	udp	private	SF	105	146	0	0	0	0 ...	254	1.00
2	0	udp	private	SF	105	146	0	0	0	0 ...	254	1.00
3	0	udp	private	SF	105	146	0	0	0	0 ...	254	1.00
4	0	udp	private	SF	105	146	0	0	0	0 ...	254	1.00
5	0	udp	private	SF	105	146	0	0	0	0 ...	255	1.00
6	0	udp	domain_u	SF	29	0	0	0	0	0 ...	3	0.30
7	0	udp	private	SF	105	146	0	0	0	0 ...	253	0.99
8	0	udp	private	SF	105	146	0	0	0	0 ...	254	1.00
9	0	tcp	http	SF	223	185	0	0	0	0 ...	255	1.00

Fig 4.13: Data Frame

Data Validation/ Cleaning/Preparing Process:

Importing the library packages and loading the given dataset. To analyze the variables, we check the data shape, data types, and evaluate missing values and duplicate entries. A validation dataset is a portion of data that is kept aside during training to estimate the model's performance while adjusting the model and its procedures. This helps in making the best use of validation and test datasets when evaluating models. Data cleaning and preparation involve renaming the dataset and dropping columns, among other steps, to analyze univariate, bivariate, and multivariate processes. The steps and techniques for data cleaning can vary depending on the dataset. The main purpose of data cleaning is to identify and remove errors and anomalies, thereby improving the quality of data for analytics and decision-making.

Exploration data analysis of visualization:

Data visualization is a key skill in applied statistics and machine learning. Statistics mainly deals with numbers and how to describe and estimate them. However, data visualization offers tools that help understand data in a different way, by showing patterns and relationships. It's useful when you're first looking at a dataset and can help you spot things like trends, bad data, unusual points, and more. If you know something about the subject area, visualizations can show important connections in a way that's easier to see and understand than just numbers or statistical results. Data visualization and exploratory data analysis are big areas on their own, so it's worth going deeper into some of the books mentioned at the end.

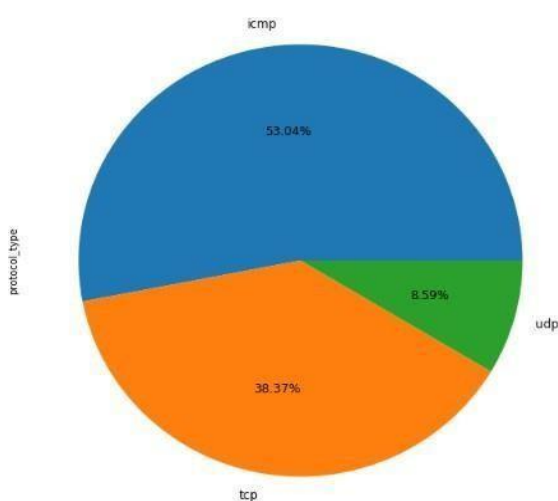


Fig 4.14: Percentage level of protocol type

Sometimes data doesn't make sense until you can see it in a visual form, like charts or graphs. The ability to quickly visualize data samples and other types of information is an important skill in applied statistics and machine learning. This will help you learn about the different types of plots you need to know when working with data in Python and how to use them to better understand your own data.

- How to create line plots for time series data and bar charts for categorical data.
- How to use histograms and box plots to summarize data distributions.
- How to use scatter plots to show the relationship between variables.

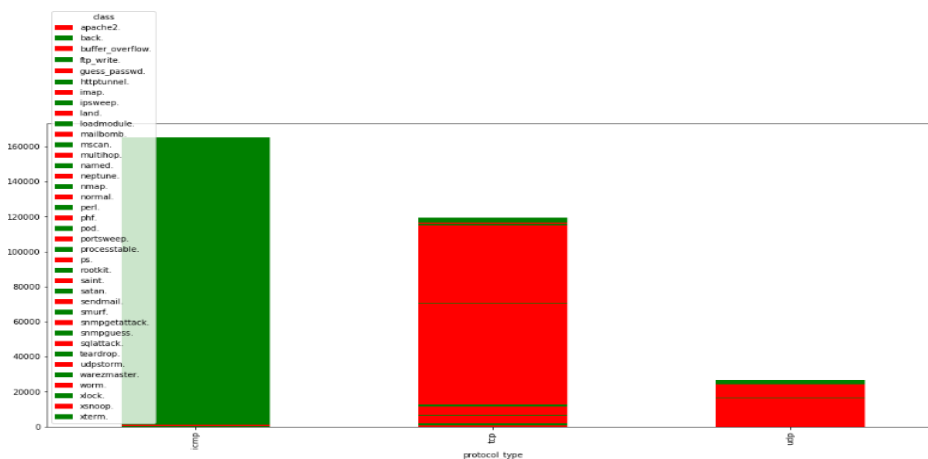


Fig 4.15: Comparison of service type and protocol type

Many machine learning algorithms are affected by the range and how values are spread out in the input data. Outliers in the data can cause problems during the training process, leading to longer training times, less accurate models, and worse overall results.

Even before building predictive models on training data, outliers can create misleading views of the data, which in turn lead to wrong interpretations. Outliers can change the average and standard deviation in statistical summaries and distort visualizations like histograms and scatterplots, making it harder to see the main part of the data. In some cases, outliers can actually be important, such as in detecting fraud or security threats. If the model can't fit properly on the training data, it's hard to be sure it will perform well on real-world data.

To fix this, we need to make sure the model learns the right patterns from the data and isn't being confused by too much noise. Cross-validation is a method where we train the model on part of the data and test it on another part to check how well it works.

Advantages of train/test split:

1. This runs K times faster than Leave One Out cross-validation because K-fold cross-validation repeats the train/test split K-times.
2. Simpler to examine the detailed results of the testing process.

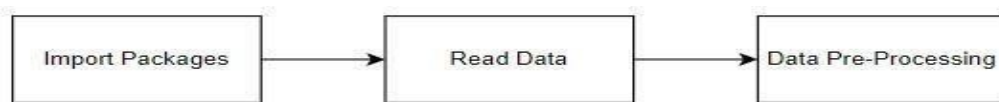
Advantages of cross-validation:

1. More accurate estimate of out-of-sample accuracy.
2. More “efficient” use of data as every observation is used for both training and testing.

Data Pre-processing:

Pre-processing is the process of changing our data before we give it to a machine learning algorithm. It's a method used to turn raw data into a clean and ready dataset. When data is collected from different sources, it's usually in a raw form, which isn't suitable for analysis. To get better results from the model, the data needs to be properly prepared. Some machine learning models require data in a specific format; for instance, the Random Forest algorithm doesn't handle null values. So, before applying Random Forest, we need to deal with those null values in the original dataset. Also, the data should be arranged in a way that allows multiple machine learning and deep learning algorithms to work on it.

MODULE DIAGRAM

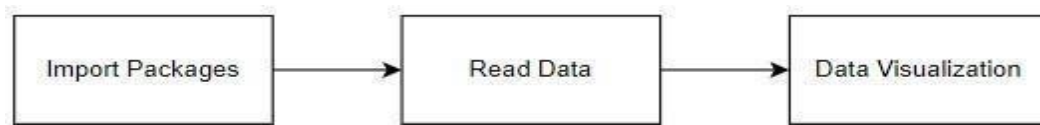


GIVEN INPUT EXPECTED OUTPUT

Input : Data

Output : Removing noisy data

MODULE DIAGRAM



GIVEN INPUT EXPECTED OUTPUT

input : Data

output : Visualized data

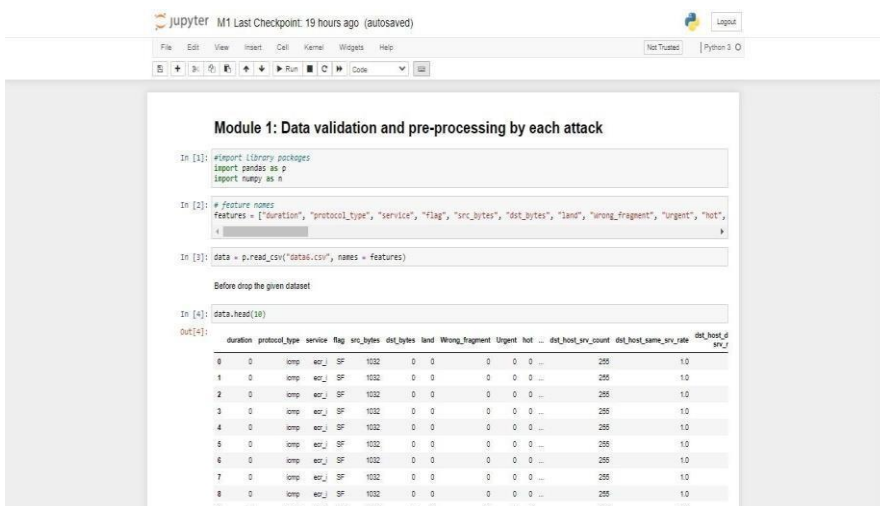


Fig 4.16: Data Validation

4.8.2 Prediction of DOS attacks (Module-02):

In computing, a denial-of-service attack, or DoS attack, is a type of cyber-attack where the attacker tries to stop a machine or network resource from being used by its intended users. They do this by disrupting the services of a computer connected to the Internet, either temporarily or permanently. This is usually done by sending a large number of unnecessary requests to the target, which overwhelms the system and stops some or all legitimate requests from being processed. In a distributed denial-of-service attack, or DDoS attack, the attack traffic comes from many different sources. This makes it much harder to stop because you can't just block one source. A DoS or DDoS attack is like a group of people crowding the entrance of a store, making it difficult for real customers to get in and causing disruption to normal business.

```

In [1]: import pandas as p

In [2]: import warnings
warnings.filterwarnings('ignore')

In [3]: # feature names
features = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
            'num_failed_logins', 'logged_in', 'num_compromised', 'num_root', 'num_file_creations', 'num_shells',
            'num_access_files', 'num_outbound_cmds', 'is_host_login', 'is_guest_login', 'count', 'srv_count', 'error_rate', 'srv_error_rate',
            'diff_srv_error_rate', 'diff_host_error_rate', 'diff_host_srv_error_rate', 'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate']

In [4]: df = data.dropna()

In [5]: df['dosland'] = df.land.map({'normal': 1, 'dosland': 0})

In [6]: df['doslandclass'] = df.dosland.map({'normal': 1, 'dosland': 0})

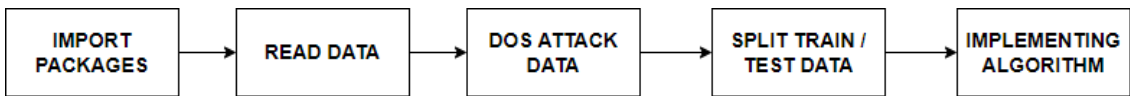
In [7]: df['dos'] = df['class'].map({'normal': 1, 'smurfattack': 0, 'named': 1, 'clock': 1, 'smurf': 1,
                                   'land': 1, 'buffer_overflow': 1, 'portscan': 1, 'pod': 1, 'apache2': 1,
                                   'perl': 1, 'subprocess': 1, 'perl': 1, 'satan': 1, 'worm': 1, 'kerberos': 1,
                                   'backdoor': 1, 'processmail': 1, 'ps': 1, 'lmap': 1, 'rootkit': 1, 'neptune': 1,
                                   'loadmodule': 1, 'lmap': 1, 'back': 1, 'httptunnel': 1, 'worm': 1,
                                   'backdoor': 1, 'ftp_write': 1, 'teardrop': 1, 'land': 1, 'sqlattack': 1,
                                   'smuggles': 1})

In [8]: from sklearn.preprocessing import LabelEncoder
var_map = {'duration': 'duration', 'protocol_type': 'service', 'flag': 'src_bytes',
            'dst_bytes': 'land', 'wrong_fragment': 'urgent', 'hot':
            'num_failed_logins', 'logged_in': 'num_compromised', 'num_shells':
            'num_attempted', 'num_root': 'num_file_creations', 'num_shells':
            'num_access_files', 'num_outbound_cmds': 'is_host_login',
            'is_guest_login': 'count', 'srv_count': 'error_rate',
            'error_rate': 'error_rate', 'srv_error_rate': 'error_rate',
            'diff_srv_error_rate': 'diff_host_error_rate', 'diff_host_error_rate':
            'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate':
            'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate':
            'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate'}

```

Fig 4.17: Prediction of DOS Attacks

MODULE DIAGRAM



4.8.3 Prediction of R2L attacks (Module-03):

These days, keeping a strong security system is really important to make sure that information is shared safely and trustfully between different organizations. However, sending secure data over the internet and other networks is always at risk of being attacked or misused. To deal with these dangers, it's very important to spot attacks early. Types of attacks like probing, Denial of Service (DoS), and Remote to Local (R2L) attacks affect many computers around the world every day. Finding these attacks and stopping them from harming computers is a big focus for researchers all over the world.

```

In [1]: import pandas as p

In [2]: # feature names
features = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
            'num_failed_logins', 'logged_in', 'num_compromised', 'num_root', 'num_file_creations', 'num_shells',
            'num_access_files', 'num_outbound_cmds', 'is_host_login', 'is_guest_login', 'count', 'srv_count', 'error_rate',
            'srv_error_rate', 'diff_srv_error_rate', 'diff_host_error_rate', 'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate']

In [3]: df = data.dropna()

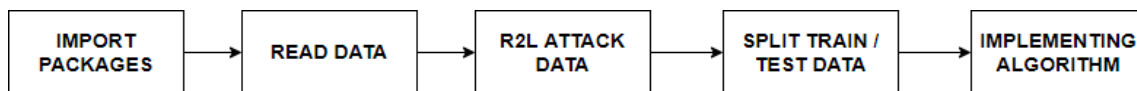
In [4]: df['r2l'] = df['class'].map({'normal': 1, 'smurfattack': 1, 'named': 1, 'clock': 1, 'smurf': 1,
                                   'land': 1, 'buffer_overflow': 1, 'portscan': 1, 'pod': 1, 'apache2': 1,
                                   'perl': 1, 'subprocess': 1, 'perl': 1, 'satan': 1, 'worm': 1, 'kerberos': 1,
                                   'backdoor': 1, 'processmail': 1, 'ps': 1, 'lmap': 1, 'rootkit': 1, 'neptune': 1,
                                   'loadmodule': 1, 'lmap': 1, 'back': 1, 'httptunnel': 1, 'worm': 1,
                                   'backdoor': 1, 'ftp_write': 1, 'teardrop': 1, 'land': 1, 'sqlattack': 1,
                                   'smuggles': 1})

In [5]: from sklearn.preprocessing import LabelEncoder
var_map = {'duration': 'duration', 'protocol_type': 'service', 'flag': 'src_bytes',
            'dst_bytes': 'land', 'wrong_fragment': 'urgent', 'hot':
            'num_failed_logins', 'logged_in': 'num_compromised', 'num_root': 'num_file_creations', 'num_shells':
            'num_attempted', 'num_root': 'num_file_creations', 'num_shells':
            'num_access_files', 'num_outbound_cmds': 'is_host_login',
            'is_guest_login': 'count', 'srv_count': 'error_rate',
            'error_rate': 'error_rate', 'srv_error_rate': 'error_rate',
            'diff_srv_error_rate': 'diff_host_error_rate', 'diff_host_error_rate':
            'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate':
            'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate':
            'diff_host_diff_host_error_rate', 'diff_host_diff_host_error_rate'}

```

Fig 4.18: Prediction of R2L Attacks

MODULE DIAGRAM



4.8.4 Prediction of U2R attacks (Module-04):

Remote to local attacks (R2L) are well-known methods used by attackers to get into a victim's machine within a network without permission. Likewise, user to root attacks (U2R) are often used to gain full control or root privileges on a machine that someone is already allowed to access. Among all U2R attacks, buffer overflow is the most common type. These attacks usually start with an attacker accessing a regular user account and then searching for passwords or other information to eventually take over the system as the root user. Detecting these kinds of attacks and stopping them is a big focus for researchers around the world.

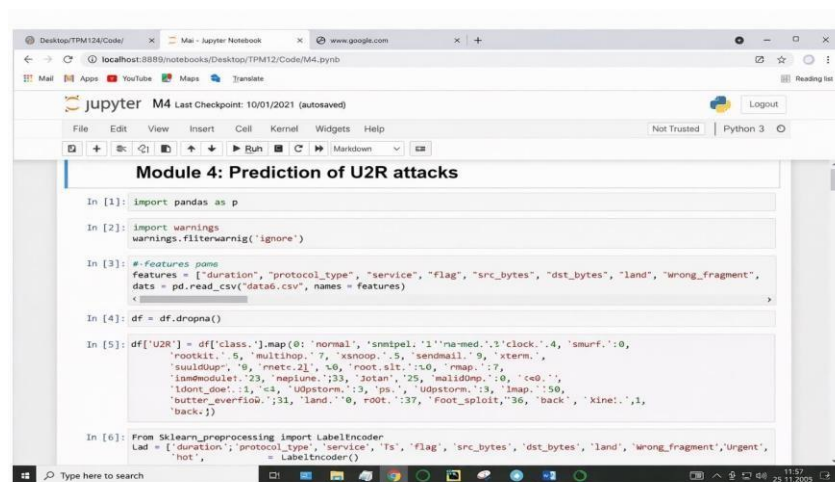
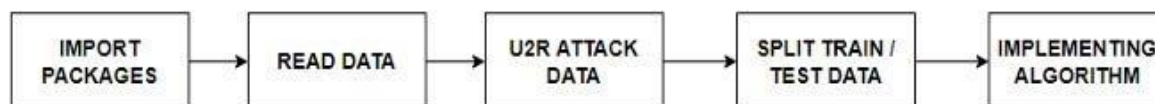


Fig 4.19: Prediction of U2R Attacks

MODULE DIAGRAM



4.8.5 Prediction of Probe attacks (Module-05):

Probing attacks are a way for hackers to get around security by looking at how a chip is made physically. Since it's an invasive attack, the attacker gets direct access to the internal wiring and connections of the device to take out sensitive data. A probe is a type of attack that's made to be noticed, so the target system can catch it and send back a clear sign or "fingerprint." The attacker then uses this information to find out where the security system is and what it can do. This kind of attack is all about

collecting information on the target machine or network to understand its structure. Learning details like open ports, IP address, hostname, and operating system can be really helpful. A Network Probe is a powerful tool for watching network traffic in real-time and quickly finding out what's causing slowdowns.

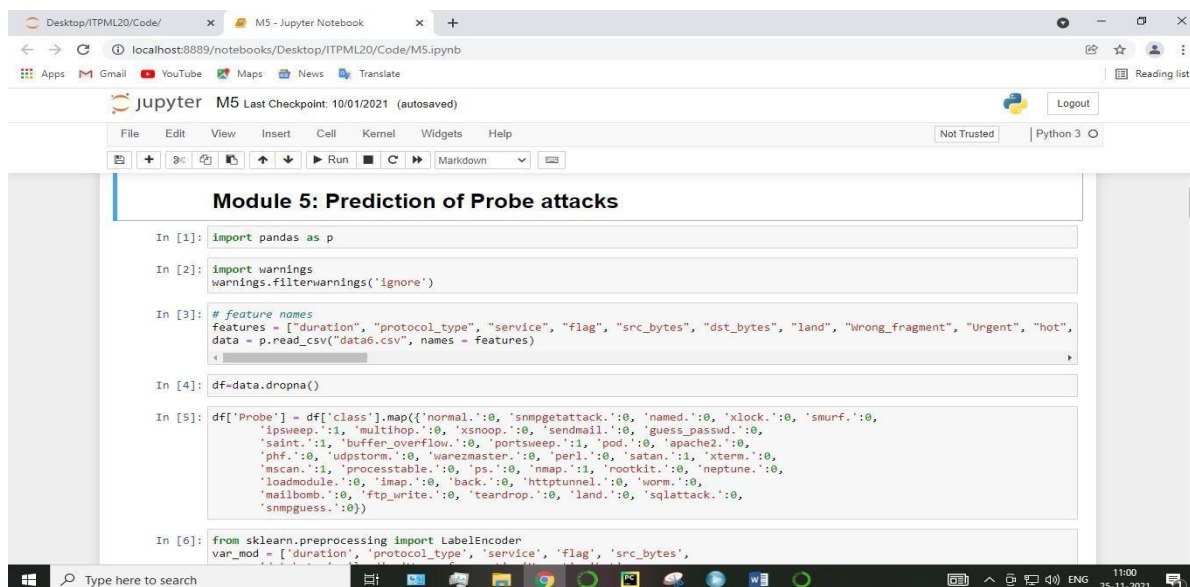
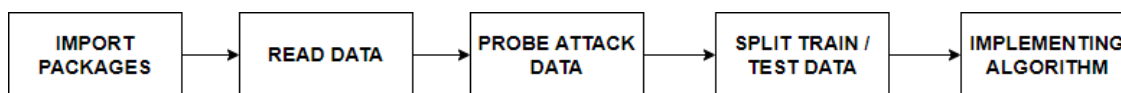


Fig 4.20: Prediction of Probe Attacks

MODULE DIAGRAM

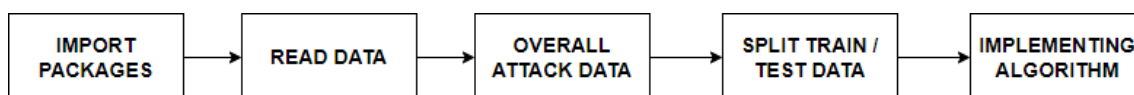


4.8.6 Prediction of overall network attacks (Module-06):

More and more, attacks are being carried out in several steps, which makes them harder to spot. These complex attacks need defenders to understand that different stages of an attack, which might take place over a longer time, are all part of the same attack. These complex attacks usually have two main parts: exploration and exploitation. During the exploration phase, attackers look for weaknesses and check a system by scanning and testing it. This is how they learn about the system's setup. The exploitation phase is when attackers use the information they've gathered to get into a system and keep access. They apply their knowledge from the exploration stage at this point. One example of a complex attack that mixes both exploration and exploitation is when a phishing attack is followed by an exfiltration attack. First, attackers will gather information about the company they want to target, for example, the names of important people. Then, they create a targeted phishing attack. This phishing attack lets the attackers access the user's system and install malware. The malware can be used to get files from the user's machine or to use that machine to attack other parts of the company's network. Usually, phishing attacks happen when someone gets an email that looks like it's from a trusted source and is tricked into clicking on a link that installs malware on their system. This malware

then opens a backdoor into the system, allowing for more complex attacks. Phishing attacks can be identified by looking at the words used in the email, like in spam, and also by checking the links included. Some features that have been helpful in detecting phishing attacks are links that include IP addresses, the age of the website linked to, and when the text of a link doesn't match the actual website it's pointing to.

MODULE DIAGRAM



4.8.7 GUI based prediction results of Network attacks (Module-07):

GUI stands for Graphical User Interface. It is the standard way users interact with devices using images like buttons and icons instead of typing commands or using text. A common example is the Microsoft operating system. The graphical user interface (GUI) is a type of user interface that lets people use electronic devices by interacting with graphical icons and audio cues, rather than text or command-based interfaces.

GUIs were created because command-line interfaces (CLIs) were difficult to learn, as they required users to type commands on a keyboard.

In a GUI, actions are typically done by directly clicking or manipulating graphical elements. GUIs are not just for computers — they're also found in many portable devices like MP3 players, gaming consoles, smartphones, and smaller home, office, and industrial controls. However, the term GUI is usually not used for other types of interfaces, such as video game interfaces (which often use head-up displays, or HUDs) or for non-flat screens like volumetric displays, since the term is associated with two-dimensional screens that display information in the traditional way studied at the Xerox Palo Alto Research Center.

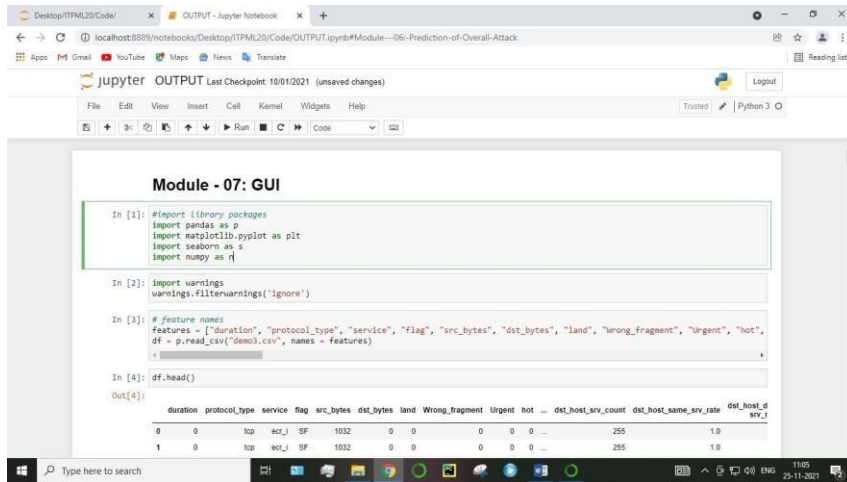
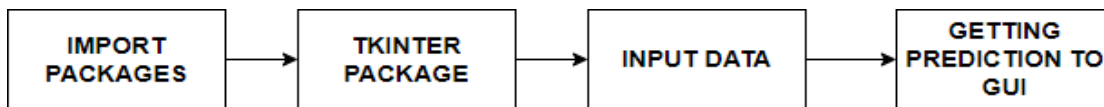


Fig 4.21: GUI based prediction

MODULE DIAGRAM



CHAPTER-5

RESULTS AND DISCUSSION, PERFORMANCE ANALYSIS

5.1 PERFORMANCE ANALYSIS

Performance Comparison of DOS Attack:

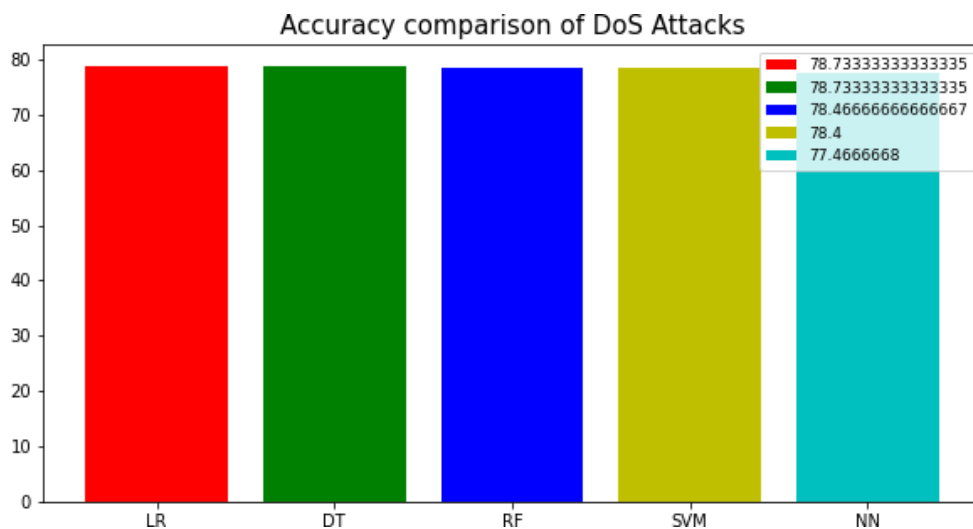


Fig 5.1: Accuracy comparison of DOS Attack

Performance Comparison of R2L Attack:

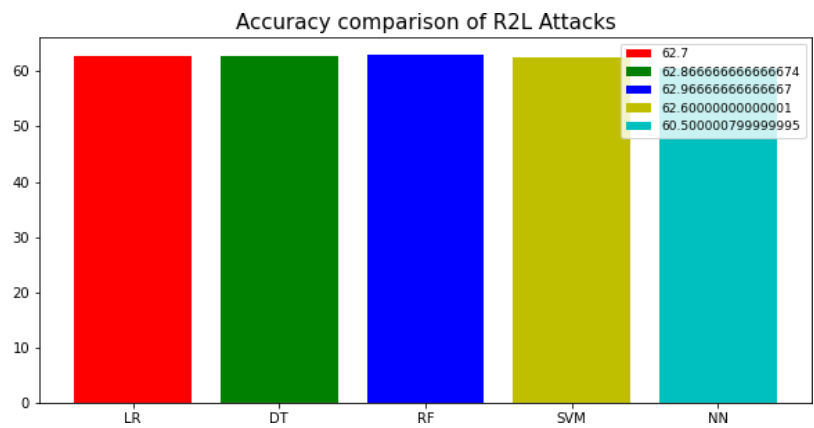


Fig 5.2: Accuracy Comparison of R2L Attacks

Performance Comparison of U2R Attack:

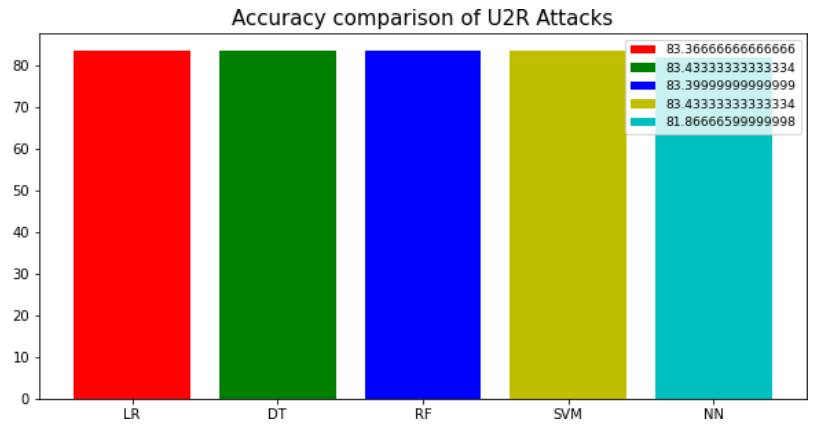


Fig 5.3: Accuracy Comparison of U2R Attacks

Performance Comparison of Probe Attack:

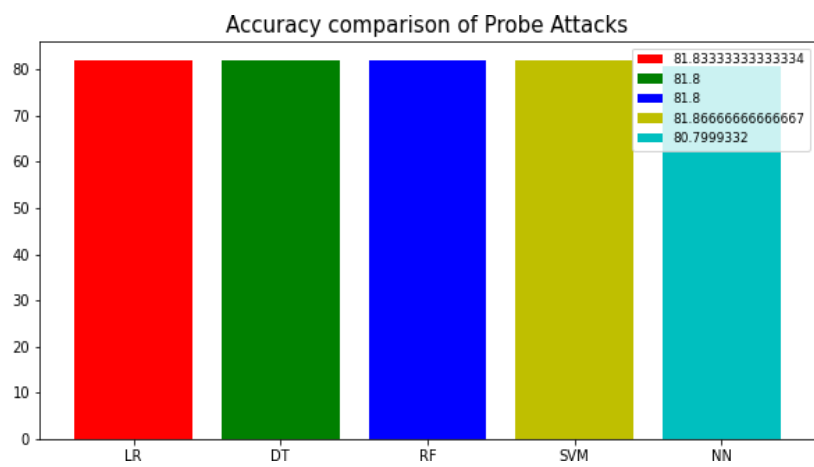


Fig 5.4: Accuracy Comparison of Probe Attacks

Performance Comparison of Overall Attacks:

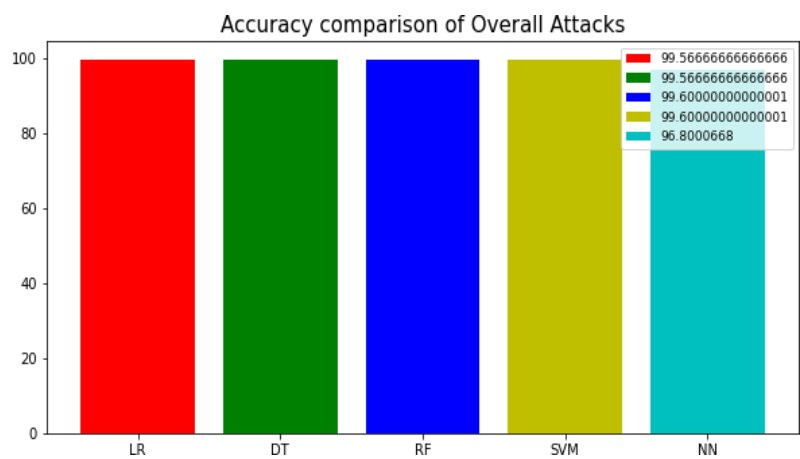


Fig 5.5: Accuracy Comparison of Overall Attacks

Deployment of GUI:

We are creating a GUI in Python using Tkinter to predict the type of attack. We have also applied some Machine Learning and Deep Learning models to it. The tkinter package, also known as the "Tk interface," is the standard Python interface for the Tcl/Tk GUI toolkit. Both Tk and tkinter are available on most Unix systems, including macOS, as well as on Windows. The GUI has experienced some performance issues when adding many widgets to the screen, like when placing a grid of buttons, which can take a few seconds. It appears that the time is due to the process of drawing the widgets on the screen. We have tried using both the grid and place geometry managers. We would appreciate any other suggestions on how to significantly improve the speed.

5.2 DISCUSSION

While discussions offer great opportunities to explore and discover new ideas, leading one can sometimes be stressful. Discussions are naturally unpredictable, and as teachers, we have to let go of some control over how information flows. Good preparation can help make discussions active and engaging without them getting out of hand. When planning a discussion, it’s important to think about more than just what students learn — also how they feel and how the physical setting affects their ability to share and exchange ideas effectively.

CHAPTER-6

SUMMARY AND CONCLUSION

6.1 SUMMARY

The scope of this project is to investigate a dataset of network connection attacks for KDD records for medical sector using machine learning technique. To identifying network connection is attacked or not.

6.2 CONCLUSION

The analytical process began with cleaning and preparing the data, handling missing values, conducting exploratory analysis, and eventually building and evaluating models. The highest accuracy on the public test set will be determined by comparing each algorithm's performance across all types of network attacks, which will help in predicting future results by identifying the best connections. This leads to some key insights regarding the detection of network attacks through each new connection. A prediction model using artificial intelligence is presented to surpass human accuracy and enable early detection. From this model, it can be concluded that area analysis and the use of machine learning techniques are valuable in creating prediction models that help network sectors shorten the diagnosis process and eliminate human errors.

6.3 FUTURE WORK

- The network sector aims to automate the detection of attack packets during real-time transfers, using details about the connection.
- I want to automate this process by displaying the prediction results through a web or desktop application.
- The goal is to optimize the implementation within an artificial intelligence environment.

REFERENCES

- [1] A. Kurakin, I. Goodfellow, and S. Bengio. (2016). "Adversarial examples in the physical world." [Online]. Available: <https://arxiv.org/abs/1607.02533>.
- [2] G. Zhang, C. Yan, X. Ji, T. Zhang, T. Zhang, and W. Xu. (2017). "Dolphin Attack: Inaudible voice commands." [Online]. Available: <https://arxiv.org/abs/1708.09537>.
- [3] P. He, W. Huang, T. He, Q. Zhu, Y. Qiao, and X. Li, "Single shot text detector with regional attention," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), Oct. 2017, pp. 3066–3074.
- [4] C. Yan, H. Xie, J. Chen, Z. Zha, X. Hao, and Y. Zhang, "A fast uyghur text detector for complex background images," IEEE Trans. Multimedia, vol. 20, no. 12, pp. 3389–3398, Dec. 2018.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in Proc. IEEE Int. Conf. Comput. Vis., 2015, pp. 1026–1034.
- [6] D. Storcheus, A. Rostamizadeh, and S. Kumar, "A survey of modern questions and challenges in feature extraction," in Proc. NIPS Workshop, Dec. 2015, pp. 1–18.
- [7] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in Proc. IEEE Symp. Secur. Privacy (SP), May 2017, pp. 3–18.
- [8] Y. Bengio and Y. LeCun, "Scaling learning algorithms towards AI," Large-Scale Kernel Mach., vol. 34, no. 5, pp. 1–41, 2007.
- [9] Y. Song, T. Kim, S. Nowozin, S. Ermon, and N. Kushman. (2017). "PixelDefend: Leveraging generative models to understand and defend against adversarial examples." [Online]. Available: <https://arxiv.org/abs/1710.10766>
- [10] A. Kurakin et al. (2018). "Adversarial attacks and defences competition." [Online]. Available: <https://arxiv.org/abs/1804.00097>
- [11] A. Fawzi, O. Fawzi, and P. Frossard, "Fundamental limits on adversarial robustness," in Proc. ICML Workshop Deep Learn., 2015, pp. 1–7.
- [12] R. M. A. Ujjan, Z. Pervez, K. Dahal, A. K. Bashir, R. Mumtaz, and J. González, "Towards sFlow and adaptive polling sampling for deep learning based DDoS detection in SDN," Future Generation Computer Systems, vol. 111, pp. 763–779, 2020, doi: 10.1016/j.future.2019.10.015.
- [13] "Software Defined Networking Definition." <https://www.opennetworking.org/sdn-definition/> (accessed March, 2, 2020).
- [14] S. Garg, K. Kaur, N. Kumar, and J. J. P. C. Rodrigues, "Hybrid Deep Learning-Based Anomaly Detection Scheme for Suspicious Flow Detection in SDN: A Social Multimedia Perspective," IEEE Transactions on Multimedia, vol. 21, no. 3, pp. 566–578, 2019, doi: 10.1109/tmm.2019.2893549.
- [15] M. Nobakht, V. Sivaraman, and R. Boreli, "A Host-Based Intrusion Detection and Mitigation Framework for Smart Home IoT Using OpenFlow," presented at the 2016 11th International Conference on Availability, Reliability and Security (ARES), 2016.